Mastering Spring Security -- Authentication, Authorization, and Enterprise Security Architecture Guide

---

Author: Senior Java Security Architect **Target Audience:** Java backend developers with 3+ years experience aiming for architect-level Spring Security expertise **Prerequisites:** Core Java, Spring Boot, REST APIs, System Design, and basic security concepts

---

Table of Contents

1. Spring Security Fundamentals & The Bridge Architecture
2. Spring Security Architecture Internals (Deep Dive)
3. Complete Authentication Flow
4. Authentication Providers & Strategy Pattern
5. UserDetails and UserDetailsService Deep Dive
6. Password Security and Encoding
7. JWT Authentication Architecture (Symmetric vs Asymmetric)
8. Spring Security with OAuth2 & OIDC
9. Role-Based Access Control (RBAC)
10. Security Filter Chain Deep Dive
11. Session-Based vs Token-Based Authentication
12. Microservices Security Architecture (Zero-Trust & mTLS)
13. Security Best Practices
14. Production Security Challenges & Scaling Security
15. Complete Spring Boot Security Project
16. Architect-Level Interview Questions (100+)
17. Hands-on Practice Exercises

Part 1: Spring Security Fundamentals & The Bridge Architecture

1.1 What is Spring Security?

Spring Security is a powerful, highly customizable authentication and access-control framework for Java applications. It is the de-facto standard for securing Spring-based applications.

Spring Security provides:

- **Authentication:** Verifying who you are (login)
- **Authorization:** Verifying what you can do (permissions)
- **Protection:** Against common security exploits (CSRF, XSS, session fixation)

Deep Concept Explanation

WHAT: Spring Security is not just a login library. It is a comprehensive security framework that provides a pluggable architecture for securing every layer of your application -- from HTTP transport to method-level business logic. It integrates seamlessly with the Servlet API, Spring MVC, Spring WebFlux, and the entire Spring ecosystem.

WHY it exists: Before Spring Security, Java developers had to manually implement security using raw Servlet Filters, custom session management, and hand-written password hashing. This led to inconsistent, buggy, and vulnerable applications. Spring Security standardizes security by providing battle-tested, production-hardened implementations of every common security pattern.

WHEN to use it: Any time you build a Spring-based application that requires:

- User login (form-based, JWT, OAuth2, LDAP)
- API endpoint protection
- Role/permission-based access control
- Protection against standard web vulnerabilities

WHERE it is used in real-world systems:

- **Banking applications:** Securing transaction APIs, implementing MFA, account lockout
- **E-commerce platforms:** Customer login, admin dashboards, payment API security
- **Healthcare systems:** HIPAA-compliant access control, audit logging
- **SaaS products:** Multi-tenant security, API key authentication for third-party integrations

How Spring Security Differs: Monolithic vs Microservices

```
MONOLITHIC ARCHITECTURE:
  Client -> [Spring Security Filter Chain] -> Controller -> Service ->
Repository

  - Single SecurityFilterChain configured in one SecurityConfig class
  - Session/JWT managed in one place
  - All endpoints secured in a single application context
  - CORS typically not needed (same origin)

MICROSERVICES ARCHITECTURE:
  Client -> [API Gateway (Spring Security)] -> Auth Service (issues JWT)
                                            -> Order Service (validates JWT)
                                            -> User Service (validates JWT)

  - Each microservice MAY have its own SecurityFilterChain
  - JWT is the standard (sessions don't scale across services)
  - API Gateway acts as the first line of defense
  - Each service can independently authorize requests
  - CORS is critical (frontend on different origin)
```

Real-World Production Scenario: Securing a Banking Login API

Consider a banking application where users log in to view their account balance:

```
Scenario: User authenticates via POST /api/auth/login

Senior Engineer's Thinking Process:
1. "The login endpoint must be public (permitAll), but ALL other endpoints must
be authenticated."
2. "Passwords must be hashed with BCrypt (never stored plain text)."
3. "After 5 failed login attempts, lock the account for 30 minutes."
4. "Return a JWT (not a session) because the mobile app and web app both call
this API."
5. "Rate limit the login endpoint to 10 requests per IP per minute to prevent
brute force."
6. "Log every authentication attempt (success/failure) for audit compliance."
```

Security Pitfalls and Common Developer Mistakes

Mistake	Why It Is Dangerous	Correct Approach

Using `permitAll()` on all endpoints during development and forgetting to change it	Exposes all APIs to the Internet	Use `authenticated()` by default, explicitly `permitAll()` only for public APIs
Disabling CSRF without understanding why	Makes session-based apps vulnerable to forgery attacks	Only disable CSRF for stateless JWT APIs, keep it for session-based apps
Storing JWT secret key in `application.yml`	Secret key leaks via Git repo	Use environment variables or Vault
Not customizing `AuthenticationEntryPoint`	Returns HTML login page for REST APIs	Return proper JSON 401 responses
Using `@Order` incorrectly with multiple filter chains	Wrong filter chain processes the request	Always test with `logging.level.org.springframework.security=DEBUG`

Interview Questions: Spring Security Fundamentals

Q: What default behavior does Spring Security add when you include the starter dependency? Every endpoint becomes protected, a default login page is generated, a random password is printed to console, CSRF is enabled, session fixation protection is active, and security headers (HSTS, X-Frame-Options, etc.) are automatically added.

Q: In a monolithic app, where does Spring Security sit in the request lifecycle? It sits BEFORE the `DispatcherServlet`. Spring Security operates as Servlet Filters registered with the embedded Tomcat/Jetty container. The request passes through the entire Security Filter Chain before reaching any `@Controller`.

Q: In a microservices app, which service should own the SecurityFilterChain? Every service that receives HTTP requests should have its own `SecurityFilterChain`. The API Gateway validates the JWT first. Each downstream microservice should ALSO validate the JWT (Zero-Trust model) or at minimum read trusted headers from the Gateway.

1.2 Architect's View: The Bridge Between Servlet and Spring

Before diving into how to write a login form, a true architect must understand *where* Spring Security sits in the request lifecycle.

Spring Security operates at the **Servlet Filter** level, long before your request hits the `DispatcherServlet` or your `@RestController`. However, standard Servlet Filters (like Tomcat's filters) do not know about Spring's Application Context (Dependency Injection, Beans, etc.).

So how does a Servlet Filter use Spring Beans like `UserDetailsService` or `JwtService`?

The Solution: DelegatingFilterProxy

Spring bridges the standard Servlet Container with the Spring Application Context using a specific filter called `DelegatingFilterProxy`.

```
Servlet Container (Tomcat)         Spring Application Context
+-------------------------+        +---------------------------+
| Http Request            |        |                           |
|      v                  |        |                           |
| +---------------------+ |        |   +-------------------+   |
| |DelegatingFilterProxy| | -----> |   | FilterChainProxy  |   |
| +---------------------+ |        |   +-------------------+   |
|      v                  |        |        | Security     |   |
| DispatcherServlet       |        |        | Filters...   |   |
+-------------------------+        +---------------------------+
```

1. **`DelegatingFilterProxy`:** This is a standard Servlet Filter registered with Tomcat/Jetty. Its only job is to intercept the request and "delegate" it to a Spring Bean. It acts as a bridge.
2. **`FilterChainProxy`:** This is the Spring Bean that `DelegatingFilterProxy` delegates the work to. `FilterChainProxy` contains the internal **Security Filter Chain(s)**.

Why this matters in production: If you get a security exception before the `DispatcherServlet`, standard Spring `@ControllerAdvice` will **not** catch it. Exceptions thrown in the filter chain must be handled within the filter chain (e.g., via `AuthenticationEntryPoint` or `AccessDeniedHandler`), not in your standard exception handlers.

Internal Working of the DelegatingFilterProxy Mechanism

When Spring Boot starts up, the following happens internally:

```
1. Spring Boot auto-configures SecurityAutoConfiguration
2. SecurityAutoConfiguration creates a SecurityFilterChain @Bean
3. Spring's SecurityFilterAutoConfiguration registers DelegatingFilterProxy with
```

```
Tomcat
4. DelegatingFilterProxy looks up the Bean named "springSecurityFilterChain"
from ApplicationContext
5. That Bean is actually FilterChainProxy
6. FilterChainProxy maintains a List<SecurityFilterChain>
7. For EACH incoming request, FilterChainProxy finds the FIRST matching chain
8. That chain's filters execute in order
```

```java
// This is conceptually what FilterChainProxy does internally:
public class FilterChainProxy extends GenericFilterBean {

    private List<SecurityFilterChain> filterChains;

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain) {
        HttpServletRequest request = (HttpServletRequest) req;

        // Find the FIRST matching SecurityFilterChain
        for (SecurityFilterChain securityChain : filterChains) {
            if (securityChain.matches(request)) {
                // Execute all filters in THIS chain
                List<Filter> filters = securityChain.getFilters();
                new VirtualFilterChain(filters, chain).doFilter(req, res);
                return;
            }
        }
        // No matching chain found = pass through to DispatcherServlet
        chain.doFilter(req, res);
    }
}
```

Real-World Production Scenario: Multiple Security Filter Chains

In a production enterprise application, you often need DIFFERENT security configurations for DIFFERENT URL patterns:

```java
// Chain 1: API endpoints use JWT (stateless)
@Bean
@Order(1)
public SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {
    return http
        .securityMatcher("/api/**")
```

```
        .csrf(csrf -> csrf.disable())
        .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated())
        .build();
}

// Chain 2: Admin console uses form login (session-based)
@Bean
@Order(2)
public SecurityFilterChain adminFilterChain(HttpSecurity http) throws Exception
{
    return http
        .securityMatcher("/admin/**")
        .formLogin(form -> form.loginPage("/admin/login"))
        .sessionManagement(s -> s.maximumSessions(1))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/admin/login").permitAll()
            .anyRequest().hasRole("ADMIN"))
        .build();
}

// Chain 3: Actuator endpoints use Basic Auth
@Bean
@Order(3)
public SecurityFilterChain actuatorFilterChain(HttpSecurity http) throws
Exception {
    return http
        .securityMatcher("/actuator/**")
        .httpBasic(Customizer.withDefaults())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/actuator/health").permitAll()
            .anyRequest().hasRole("OPS"))
        .build();
}
```

Senior Engineer's Thinking Process:

- "My REST APIs are consumed by React SPAs and mobile apps -- they need stateless JWT."
- "My admin dashboard is a Thymeleaf app -- it needs session-based form login."
- "My actuator endpoints are called by Prometheus -- they need simple Basic Auth."
- "The `@Order` annotation ensures the API chain is checked FIRST because it is the most specific."

Performance and Scalability Considerations for FilterChainProxy

Concern	Impact	Mitigation
Too many filters in the chain	Each request passes through ALL filters, even if they don't apply	Use `shouldNotFilter()` in your custom `OncePerRequestFilter` to skip non-applicable requests
Heavy DB queries inside `UserDetailsService`	Called on every JWT-authenticated request if not careful	Cache `UserDetails` or extract user info directly from the JWT claims instead of hitting the DB
Multiple `SecurityFilterChain` beans without proper `@Order`	Wrong chain matches, leading to 401/403 errors	Always use `securityMatcher()` to scope chains and test with security debug logs

---

1.3 Common Security Threats

Threat	Description	Spring Security Defense
SQL Injection	Malicious SQL in input	JPA/Hibernate parameterized queries
Cross-Site Scripting (XSS)	Injecting scripts into pages	Response header security (CSP)
CSRF	Forged requests from other sites	CSRF token validation / SameSite cookies
Session Hijacking	Stealing session cookies	Session fixation protection, HttpOnly cookies
Brute Force	Guessing passwords repeatedly	Account lockout, rate limiting, CAPTCHA
Credential Stuffing	Using leaked credentials	Password encoding (Argon2/BCrypt), MFA
Man-in-the-Middle	Intercepting network traffic	HTTPS enforcement (HSTS header)

Deep Dive: How Each Threat Works in Production

SQL Injection -- Real Scenario: An attacker enters `' OR 1=1 --` in the login email field. If the backend uses string concatenation for SQL (`"SELECT * FROM users WHERE email = '" + email + "'"`), the attacker bypasses authentication. Spring Data JPA prevents this because all queries use parameterized prepared statements internally.

CSRF -- Real Scenario: A user is logged into their bank at `secure-bank.com` (session cookie stored). The user visits `evil-site.com`, which contains a hidden form that auto-submits `POST secure-bank.com/transfer?to=attacker&amount=10000`. The browser automatically attaches the bank's session cookie. The bank processes the transfer!

Spring Security prevents this by embedding a random `_csrf` token in every form. The attacker's evil form doesn't have this token, so the request is rejected.

Brute Force -- Real Scenario: An attacker runs a script testing 1 million common passwords against the admin email. Without rate limiting, the server happily processes all 1 million requests. With Spring Security + a `LoginAttemptService`:

```
@Service
public class LoginAttemptService {

    private static final int MAX_ATTEMPTS = 5;
    private final LoadingCache<String, Integer> attemptsCache;

    public LoginAttemptService() {
        // Tracks failed login attempts per email, expires after 30 minutes
        attemptsCache = CacheBuilder.newBuilder()
            .expireAfterWrite(30, TimeUnit.MINUTES)
            .build(CacheLoader.from(() -> 0));
    }

    public void loginFailed(String email) {
        int attempts = attemptsCache.getUnchecked(email);
        attemptsCache.put(email, attempts + 1);
    }

    public boolean isBlocked(String email) {
        return attemptsCache.getUnchecked(email) >= MAX_ATTEMPTS;
    }

    public void loginSucceeded(String email) {
        attemptsCache.invalidate(email);
    }
}

// Usage in CustomUserDetailsService:
@Override
public UserDetails loadUserByUsername(String email) {
    if (loginAttemptService.isBlocked(email)) {
        throw new LockedException("Account temporarily locked due to too many
failed attempts.");
```

```
    }
    // ... normal flow
}
```

Monolithic vs Microservices: Where Threats Manifest Differently

Threat	Monolithic Impact	Microservices Impact
CSRF	Single server, session cookies make it HIGH risk	Stateless JWT tokens in headers make it LOW risk
Session Hijacking	Stealing JSESSIONID gives access to everything	No sessions; attacker must steal the JWT, which expires quickly
Brute Force	Rate limiting on single server is easy	Must rate limit at the API Gateway level; individual services can't see total request volume
SQL Injection	Affects one DB	Could affect multiple service databases if shared patterns are used
Token Theft	Less relevant (sessions)	Critical! Stolen JWT = access to ALL microservices until expiry

---

Part 2: Spring Security Architecture Internals (Deep Dive)

2.1 Architecture Diagram

```
HTTP Request
     |
     v
+-------------------------------------------------------------------+
|                   SECURITY FILTER CHAIN                           |
|                                                                    |
|  +-------------------+                                            |
|  | Security Context  |  Loads SecurityContext from session/token  |
|  | Persistence Filter|  (ThreadLocal storage)                     |
|  +-------------------+                                            |
|           |                                                        |
|  +-------------------+                                            |
|  | Authentication    |  UsernamePasswordAuthenticationFilter      |
|  | Processing Filter |  OR BearerTokenAuthenticationFilter        |
|  +-------------------+                                            |
|           |                                                        |
|           v                                                        |
|  +-------------------+     +------------------------+             |
|  | Authentication    | --> | Authentication         |             |
|  | Manager           |     | Provider               |             |
|  | (ProviderManager) |     | (DaoAuthProvider)      |             |
|  +-------------------+     +------------------------+             |
|                                     |                              |
|                            +--------+--------+                    |
|                            |                 |                    |
|                   +--------v-----+   +-------v--------+          |
|                   |UserDetails   |   |Password        |          |
|                   |Service       |   |Encoder         |          |
|                   |(loads user)  |   |(verifies pwd)  |          |
|                   +--------------+   +----------------+          |
|           |                                                        |
|  +-------------------+                                            |
|  | Exception         |  Handles AuthenticationException           |
|  | Translation Filter|  and AccessDeniedException                 |
|  +-------------------+                                            |
|           |                                                        |
|  +-------------------+                                            |
```

```
|  | Authorization     |  Checks roles/authorities                  |
|  | Filter            |  for the requested URL                    |
|  +-------------------+                                            |
+-------------------------------------------------------------------+
     |
     v
Controller (if authorized)
```

Internal Working: Step-by-Step Request Lifecycle

Understanding the exact internal sequence is what separates a developer from an architect. Here is what happens inside Spring Security for EVERY single HTTP request:

```
1. Tomcat receives raw HTTP request on port 8080
   |
2. Tomcat hands request to DelegatingFilterProxy (registered in servlet context)
   |
3. DelegatingFilterProxy delegates to FilterChainProxy (Spring Bean)
   |
4. FilterChainProxy iterates through registered SecurityFilterChain beans
   |  - Finds first chain where securityMatcher matches the request URL
   |
5. SecurityContextPersistenceFilter runs FIRST
   |  - For session-based: loads SecurityContext from HttpSession
   |  - For stateless: SecurityContext is empty (requires JWT filter to populate
it)
   |
6. CsrfFilter validates anti-forgery token (if enabled)
   |  - For stateless APIs: this filter is typically disabled
   |
7. Authentication Filters run (e.g., UsernamePasswordAuthenticationFilter)
   |  - For JWT: your custom JwtAuthenticationFilter runs here
   |  - Extracts credentials -> creates unauthenticated Authentication token
   |  - Calls AuthenticationManager.authenticate()
   |    -> ProviderManager loops through AuthenticationProviders
   |    -> DaoAuthenticationProvider calls UserDetailsService
   |    -> PasswordEncoder.matches() is called
   |  - If successful: creates authenticated Authentication token
   |  - Stores in SecurityContextHolder.getContext().setAuthentication()
   |
8. ExceptionTranslationFilter wraps the remaining chain in try-catch
   |  - Catches AuthenticationException -> triggers AuthenticationEntryPoint
(401)
   |  - Catches AccessDeniedException -> triggers AccessDeniedHandler (403)
   |
9. AuthorizationFilter checks URL patterns from authorizeHttpRequests()
```

```
   |  - Evaluates: permitAll(), authenticated(), hasRole(), hasAuthority()
   |  - If denied: throws AccessDeniedException (caught by step 8)
   |
10. If all filters pass: request reaches DispatcherServlet -> @Controller
```

Real-World Production Debugging Scenario

Problem: A developer reports: "My API returns 403 Forbidden even though the user has ROLE_ADMIN."

Senior Engineer's Debugging Process:

```
Step 1: Enable security debug logging
        logging.level.org.springframework.security=DEBUG

Step 2: Check the logs for the request:
        - "Checking authorization for GET /api/admin/users"
        - "Principal: null" --> The user is NOT authenticated!
        - Root cause: The JwtFilter is not running before the
AuthorizationFilter

Step 3: Verify filter registration:
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)

Step 4: If Principal IS populated but still 403:
        - Check: User has "ROLE_ADMIN" but config uses hasAuthority("ADMIN")
        - Fix: Either use hasRole("ADMIN") (auto-prefixes ROLE_)
                or store "ROLE_ADMIN" as the authority string
```

2.2 SecurityContextHolder & Threading Strategies

Once a user is authenticated, their details are stored in the `SecurityContext`. But how does Spring ensure that user A doesn't see user B's auth details if they are making requests at the same exact millisecond?

ThreadLocal storage. The `SecurityContextHolder` utilizes the concept of `ThreadLocal`, ensuring that the `SecurityContext` is bound strictly to the current executing Java Thread (which Tomcat assigns per incoming HTTP request).

```java
// Retrieving the current user anywhere in the request thread
SecurityContext context = SecurityContextHolder.getContext();
```

```
Authentication auth = context.getAuthentication();
String username = auth.getName();
```

Deep Concept Explanation: ThreadLocal Under the Hood

WHAT: `ThreadLocal` is a Java mechanism that gives each thread its own isolated copy of a variable. When Thread A sets a `ThreadLocal` value, Thread B cannot see it. Spring Security uses this so that each HTTP request thread has its own isolated `SecurityContext`.

WHY: In a web server like Tomcat, hundreds of threads handle concurrent requests. Without `ThreadLocal`, all threads would share the same `SecurityContext`, meaning User A's login could overwrite User B's context, leading to catastrophic security bugs (one user seeing another user's data).

HOW it works internally:

```
// Simplified SecurityContextHolder internals
public class SecurityContextHolder {

    private static final ThreadLocal<SecurityContext> contextHolder = new
ThreadLocal<>();

    public static SecurityContext getContext() {
        SecurityContext ctx = contextHolder.get();
        if (ctx == null) {
            ctx = new SecurityContextImpl();
            contextHolder.set(ctx);
        }
        return ctx;
    }

    public static void setContext(SecurityContext context) {
        contextHolder.set(context);
    }

    public static void clearContext() {
        contextHolder.remove(); // CRITICAL: prevents memory leaks in thread
pools!
    }
}
```

Advanced Architect Concept: Asynchronous Context Propagation

What happens if your Controller spawns a new thread using `@Async` or `CompletableFuture`? **The new thread will lose the SecurityContext because it is a different Thread.**

To solve this, Spring Security provides multiple strategies:

1. `MODE_THREADLOCAL` **(Default):** Context is local to the current thread. Spawning a new thread loses the context.
2. `MODE_INHERITABLETHREADLOCAL` **:** Context is copied from the parent thread to child threads. Works fine if you spawn threads manually, but fails with Thread Pools where threads are reused.
3. `MODE_GLOBAL` **:** Context applies to the entire JVM (useful for swing/desktop applications, terrible for web servers).

The Architect's Solution for Thread Pools: Wrap your Thread Pools with `DelegatingSecurityContextExecutorService` so Spring automatically copies the context into the background thread pool task.

```java
@Bean
public Executor asyncExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(5);
    executor.setMaxPoolSize(10);
    executor.setQueueCapacity(25);
    executor.setThreadNamePrefix("async-security-");
    executor.initialize();
    // Wrap the executor so SecurityContext propagates to @Async methods!
    return new DelegatingSecurityContextExecutor(executor);
}
```

Real-World Production Scenario: Audit Logging in Background Threads

```
Scenario: After a user places an order, your system sends a confirmation email
          asynchronously using @Async. The email service needs to log WHO placed
          the order for audit compliance.

Problem Without Context Propagation:
  @Async
  public void sendOrderConfirmation(Long orderId) {
      String username = SecurityContextHolder.getContext()
          .getAuthentication().getName();
      // THROWS NullPointerException! SecurityContext is empty in this thread!
  }
```

```
Solution: Use DelegatingSecurityContextExecutor (configured above)
  Now the @Async method inherits the SecurityContext from the calling thread.
  The audit log correctly records the username.
```

Monolithic vs Microservices: SecurityContext Behavior

Aspect	Monolithic	Microservices
Context source	Loaded from HttpSession or JWT on each request	Loaded from JWT only (no shared sessions)
Cross-service context	N/A (single process)	JWT must be forwarded via FeignClient headers. Each service rebuilds its own SecurityContext from the JWT.
@Async threads	Use `DelegatingSecurityContextExecutor`	Same solution. Additionally, when emitting Kafka events, serialize the JWT into the message header so consumers can rebuild context.

Security Pitfalls: SecurityContextHolder

Mistake	Consequence	Fix
Calling `SecurityContextHolder.getContext()` inside a Kafka consumer	Returns null (Kafka threads have no HTTP context)	Explicitly set a system-level `Authentication` in the Kafka listener method
Not calling `SecurityContextHolder.clearContext()` at end of request	Memory leak in thread pools; next request on same thread sees stale user	Spring automatically clears it via `SecurityContextPersistenceFilter`, but custom threads must clear manually
Using `MODE_INHERITABLETHREADLOCAL` with a fixed-size thread pool	Stale context from previous task leaks into new tasks	Always use `DelegatingSecurityContextExecutor` instead

---

Part 3: Complete Authentication Flow

3.1 Step-by-Step State Transition Flow

```
Step 1: Client sends login request
        POST /api/auth/login
        { "email": "user@mail.com", "password": "secret123" }
              |
              v
Step 2: Request hits Security Filter Chain
        UsernamePasswordAuthenticationFilter intercepts
        Creates UsernamePasswordAuthenticationToken (unauthenticated)
        auth.isAuthenticated() == false
              |
              v
Step 3: AuthenticationManager receives token
        Delegates to configured AuthenticationProvider(s)
              |
              v
Step 4: DaoAuthenticationProvider processes authentication
        Calls UserDetailsService.loadUserByUsername("user@mail.com")
              |
              v
Step 5: UserDetailsService loads user from database
        Returns UserDetails object with:
        - username, passwordHash, authorities, accountStatus
              |
              v
Step 6: Password validation
        PasswordEncoder.matches("secret123", "$2a$10$...")
        BCrypt compares raw password with stored hash
              |
              v
Step 7: If valid: Create authenticated token
        UsernamePasswordAuthenticationToken is recreated
        auth.isAuthenticated() == true
        Contains: principal (UserDetails), authorities (ROLE_USER)
              |
              v
Step 8: SecurityContextHolder stores authentication
        SecurityContextHolder.getContext().setAuthentication(authToken)
              |
              v
```

```
Step 9: Generate response (JWT token or session)
        Return JWT to client for stateless auth
        Or create HTTP session for stateful auth
```

Deep Concept Explanation

WHAT: The Authentication flow is the complete journey of a user's credentials from the HTTP request body to a verified, trusted `Authentication` object stored in the `SecurityContext`. Every subsequent request uses this stored authentication to authorize access.

WHY this step-by-step matters: If you skip step 6 (password validation), anyone can log in with any password. If you skip step 8 (storing in SecurityContext), the user is authenticated but Spring doesn't know it -- every subsequent API call returns 401.

Internal Working: The Authentication Token State Machine

The `UsernamePasswordAuthenticationToken` has TWO distinct states. Understanding these is critical for custom implementations:

```java
// STATE 1: UNAUTHENTICATED (created by the filter)
// This constructor sets authenticated = false
UsernamePasswordAuthenticationToken unauthToken =
    new UsernamePasswordAuthenticationToken("user@mail.com", "secret123");
// unauthToken.isAuthenticated() == false
// unauthToken.getPrincipal() == "user@mail.com" (just a string)
// unauthToken.getCredentials() == "secret123" (raw password)

// STATE 2: AUTHENTICATED (created by the provider after verification)
// This constructor sets authenticated = true AND accepts authorities
UsernamePasswordAuthenticationToken authToken =
    new UsernamePasswordAuthenticationToken(
        userDetails,           // principal = full UserDetails object
        null,                  // credentials = null (CLEARED for security!)
        userDetails.getAuthorities()  // ROLE_USER, ROLE_ADMIN etc.
    );
// authToken.isAuthenticated() == true
// authToken.getPrincipal() == CustomUserDetails object
// authToken.getCredentials() == null (password wiped from memory!)
// authToken.getAuthorities() == [ROLE_USER]
```

Why are credentials set to null after authentication? Once the password is verified, there is no reason to keep it in memory. If the server's memory is dumped (e.g., via a heap dump attack or logging mistake), the raw password would be exposed. Setting credentials to `null` is a defense-in-depth practice.

Real-World Production Scenario: Custom Login Flow for REST APIs

In most production REST APIs, you do NOT use `UsernamePasswordAuthenticationFilter` directly. Instead, you create a custom `AuthController` that manually drives the authentication flow:

```
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthenticationManager authManager;
    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@RequestBody @Valid LoginRequest
request) {

        // Step 1: Create unauthenticated token
        // Step 2: AuthenticationManager validates via
DaoAuthenticationProvider
        // Step 3: If invalid, BadCredentialsException is thrown automatically
        authManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                request.getEmail(), request.getPassword()));

        // Step 4: If we reach here, credentials are valid
        UserDetails user =
userDetailsService.loadUserByUsername(request.getEmail());

        // Step 5: Generate JWT tokens
        String accessToken = jwtService.generateAccessToken(user);
        String refreshToken = jwtService.generateRefreshToken(user);

        return ResponseEntity.ok(new AuthResponse(accessToken, refreshToken));
    }
}
```

Senior Engineer's Thinking Process:

1. "I don't rely on the Spring Security filter for login because I need full control over the JSON response format."

2. "I call `authManager.authenticate()` which internally calls `DaoAuthenticationProvider -> UserDetailsService -> PasswordEncoder`."
3. "If the password is wrong, Spring throws `BadCredentialsException` which my `@ControllerAdvice` catches and returns a 401 JSON response."
4. "After authentication, I generate BOTH an access token (15 min) and a refresh token (7 days)."

Monolithic vs Microservices: Authentication Flow Differences

```
MONOLITHIC APPLICATION:
  Client -> POST /api/auth/login -> AuthController
          -> AuthenticationManager -> UserDetailsService (same DB)
          -> PasswordEncoder.matches()
          -> Generate JWT -> Return to Client

  Everything happens in ONE JVM. The UserDetailsService directly queries
  the same database that stores the user table.

MICROSERVICES ARCHITECTURE:
  Client -> POST /api/auth/login -> API Gateway (routes to Auth Service)

  Auth Service:
          -> AuthController -> AuthenticationManager
          -> UserDetailsService -> User DB (Auth Service's own DB)
          -> Generate JWT (signed with PRIVATE key)
          -> Return JWT to Client via Gateway

  Subsequent API Call:
  Client -> GET /api/orders -> API Gateway (validates JWT with PUBLIC key)
          -> Forwards to Order Service (also validates JWT)
          -> Order Service extracts userId from JWT claims
          -> Queries Order DB for that user's orders

  Key Difference: The Order Service does NOT call UserDetailsService.
  It trusts the JWT claims. This is stateless authentication.
```

Part 4: Authentication Providers & Strategy Pattern

4.1 The Strategy Design Pattern in AuthenticationManager

The `AuthenticationManager` does not actually authenticate anything. It uses the **Strategy Design Pattern** via its default implementation, `ProviderManager`.

The `ProviderManager` maintains a list of `AuthenticationProvider`s. When an authentication request comes in, the `ProviderManager` loops through the providers asking, "*Can you authenticate this type of token?*" The first provider that says "Yes" performs the actual logic.

```
// Internal concept of how ProviderManager works
for (AuthenticationProvider provider : this.providers) {
    if (provider.supports(authentication.getClass())) {
        try {
            return provider.authenticate(authentication);
        } catch (AuthenticationException e) {
            // Keep trying other providers or throw
        }
    }
}
```

Architectural Benefit: This allows your application to support **Database Login (DaoAuthProvider)**, **LDAP Base Authentication**, and **OAuth2/Social Login** simultaneously in the exact same application.

Deep Concept Explanation: Why the Strategy Pattern?

WHAT: The Strategy Pattern is a behavioral design pattern that defines a family of algorithms (authentication methods), encapsulates each one, and makes them interchangeable. The `ProviderManager` is the Context; each `AuthenticationProvider` is a concrete Strategy.

WHY Spring chose this pattern: In enterprise environments, a single application might need to authenticate users from 3+ different sources:

- **Employees** authenticate via LDAP (Active Directory)
- **Customers** authenticate via database credentials
- **Third-party integrators** authenticate via API Keys

- **SSO users** authenticate via OAuth2/Keycloak

Without the Strategy Pattern, you'd need a massive `if-else` ladder. With it, you simply register multiple providers and Spring iterates through them.

HOW the `supports()` method works: Each provider declares which type of `Authentication` token it can handle. When `ProviderManager` receives a token, it asks each provider: "Do you support `UsernamePasswordAuthenticationToken.class`?" Only matched providers attempt authentication.

```
ProviderManager receives: UsernamePasswordAuthenticationToken

Provider 1: DaoAuthenticationProvider
  -> supports(UsernamePasswordAuthenticationToken.class)? YES
  -> authenticate() -> loads user from DB, checks password
  -> If success: returns authenticated token
  -> If BadCredentials: ProviderManager tries next provider

Provider 2: LdapAuthenticationProvider
  -> supports(UsernamePasswordAuthenticationToken.class)? YES
  -> authenticate() -> tries LDAP server
  -> If success: returns authenticated token

Provider 3: ApiKeyAuthenticationProvider
  -> supports(UsernamePasswordAuthenticationToken.class)? NO
  -> supports(ApiKeyAuthenticationToken.class)? YES
  -> SKIPPED for this request (wrong token type)
```

Real-World Production Scenario: Hybrid Authentication System

```
Enterprise Scenario: A hospital management system where:
- Doctors authenticate via hospital's Active Directory (LDAP)
- Patients authenticate via email/password stored in PostgreSQL
- IoT medical devices authenticate via API keys

Solution: Register three AuthenticationProviders in ProviderManager

@Configuration
public class MultiProviderConfig {

    @Bean
    public AuthenticationManager authManager(
            DaoAuthenticationProvider dbProvider,
```

```
            LdapAuthenticationProvider ldapProvider,
            ApiKeyAuthenticationProvider apiKeyProvider) {
        return new ProviderManager(
            List.of(dbProvider, ldapProvider, apiKeyProvider)
        );
    }
}
```

4.2 DaoAuthenticationProvider (Most Common)

Authenticates using a `UserDetailsService` and `PasswordEncoder`.

```java
@Bean
public AuthenticationProvider daoAuthProvider() {
    DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
    provider.setUserDetailsService(userDetailsService);
    provider.setPasswordEncoder(passwordEncoder());
    return provider;
}
```

Internal Working of DaoAuthenticationProvider

```
1. Receives UsernamePasswordAuthenticationToken
2. Extracts username from token.getPrincipal()
3. Calls UserDetailsService.loadUserByUsername(username)
4. If user not found: throws UsernameNotFoundException
5. Checks UserDetails status flags:
   - isEnabled() == false? throws DisabledException
   - isAccountNonLocked() == false? throws LockedException
   - isAccountNonExpired() == false? throws AccountExpiredException
6. Calls PasswordEncoder.matches(rawPassword, storedHash)
   - If mismatch: throws BadCredentialsException
7. Checks isCredentialsNonExpired()
   - If expired: throws CredentialsExpiredException
8. Creates fully authenticated UsernamePasswordAuthenticationToken
9. Returns to ProviderManager -> SecurityContextHolder stores it
```

Security Pitfall: Timing Attacks on User Enumeration

Problem: If `loadUserByUsername()` throws `UsernameNotFoundException`, the response is faster than when the username exists but the password is wrong (because BCrypt matching takes ~100ms). An attacker can measure response times to determine which emails are registered.

Spring Security's Defense: `DaoAuthenticationProvider` has a built-in mitigation. If the user is not found, it STILL runs `PasswordEncoder.matches()` against a dummy hash to equalize response times. This behavior is automatic.

4.3 Custom Provider Implementation (e.g. API Key)

In microservices, you might authenticate based on an `X-API-Key` header instead of a password.

```
@Component
public class ApiKeyAuthenticationProvider implements AuthenticationProvider {

    @Autowired
    private ApiKeyRepository apiKeyRepository;

    @Override
    public Authentication authenticate(Authentication authentication) throws
AuthenticationException {
        String apiKey = (String) authentication.getCredentials();

        ApiKeyEntity entity = apiKeyRepository.findByKey(apiKey)
            .orElseThrow(() -> new BadCredentialsException("Invalid API Key"));

        return new ApiKeyAuthenticationToken(
            entity.getOwnerId(),
            null,
            List.of(new SimpleGrantedAuthority("ROLE_API_CLIENT"))
        );
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return
ApiKeyAuthenticationToken.class.isAssignableFrom(authentication);
    }
}
```

Step-by-Step: Complete API Key Authentication Implementation

To make the above work in production, you also need a custom filter that extracts the API key from the request header:

```
// Step 1: Create a custom Authentication token
public class ApiKeyAuthenticationToken extends AbstractAuthenticationToken {

    private final String apiKey;
    private final Object principal;

    // Unauthenticated constructor (filter creates this)
    public ApiKeyAuthenticationToken(String apiKey) {
        super(null);
        this.apiKey = apiKey;
        this.principal = null;
        setAuthenticated(false);
    }

    // Authenticated constructor (provider creates this)
    public ApiKeyAuthenticationToken(Object principal, String apiKey,
            Collection<? extends GrantedAuthority> authorities) {
        super(authorities);
        this.principal = principal;
        this.apiKey = apiKey;
        setAuthenticated(true);
    }

    @Override public Object getCredentials() { return apiKey; }
    @Override public Object getPrincipal() { return principal; }
}

// Step 2: Create the filter that extracts API key from header
@Component
public class ApiKeyAuthenticationFilter extends OncePerRequestFilter {

    @Autowired private AuthenticationManager authManager;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
            HttpServletResponse response, FilterChain chain)
            throws ServletException, IOException {

        String apiKey = request.getHeader("X-API-Key");
        if (apiKey != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
            try {
                Authentication auth = authManager.authenticate(
```

```
                    new ApiKeyAuthenticationToken(apiKey));
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (AuthenticationException e) {
                // Invalid API key - continue chain, will be caught by
AuthorizationFilter
            }
        }
        chain.doFilter(request, response);
    }
}

// Step 3: Register the filter in SecurityConfig
@Bean
public SecurityFilterChain apiKeyChain(HttpSecurity http) throws Exception {
    return http
        .securityMatcher("/api/external/**")
        .addFilterBefore(apiKeyFilter,
UsernamePasswordAuthenticationFilter.class)
        .authorizeHttpRequests(auth -> auth.anyRequest().hasRole("API_CLIENT"))
        .build();
}
```

Part 5: UserDetails and UserDetailsService Deep Dive

5.1 Custom UserDetails Implementation

Your database User domain entity should NOT implement `UserDetails` directly. This violates the Single Responsibility Principle. Instead, create a wrapper.

Deep Concept Explanation

WHAT: `UserDetails` is Spring Security's representation of a user. It is NOT your JPA entity. It is a security-specific contract that tells Spring: "Here is the username, the hashed password, the list of roles/authorities, and whether the account is active/locked/expired."

WHY create a wrapper instead of implementing UserDetails on the JPA entity:

1. **Single Responsibility:** Your `User` entity should model business data (name, address, phone). Security concerns (authorities, account locking) should be separate.
2. **Serialization issues:** `UserDetails` is stored in `SecurityContext`. If your JPA entity has lazy-loaded relationships (e.g., `@OneToMany(fetch = LAZY)`), serialization fails with `LazyInitializationException`.
3. **Flexibility:** You might change your DB schema but don't want to break Spring Security's contract.

```java
@Getter
public class CustomUserDetails implements UserDetails {

    private final Long id;
    private final String email;
    private final String password;
    private final Collection<? extends GrantedAuthority> authorities;
    private final boolean active;
    private final boolean locked;

    public CustomUserDetails(User user) {
        this.id = user.getId();
        this.email = user.getEmail();
        this.password = user.getPasswordHash();
        this.authorities = user.getRoles().stream()
```

```
            .map(role -> new SimpleGrantedAuthority("ROLE_" + role.getName()))
            .collect(Collectors.toList());
        this.active = user.isActive();
        this.locked = user.isLocked();
    }

    @Override public String getUsername() { return email; }
    @Override public String getPassword() { return password; }
    @Override public boolean isEnabled() { return active; }
    @Override public boolean isAccountNonLocked() { return !locked; }
    @Override public boolean isAccountNonExpired() { return true; }
    @Override public boolean isCredentialsNonExpired() { return true; }
    @Override public Collection<? extends GrantedAuthority> getAuthorities() {
        return authorities;
    }
}
```

Real-World Production Scenario: Multi-Tenant UserDetails

In a SaaS application, each user belongs to a tenant (company). The `CustomUserDetails` must carry the tenant ID so that every DB query is scoped to the correct tenant:

```
@Getter
public class TenantAwareUserDetails implements UserDetails {

    private final Long userId;
    private final Long tenantId;      // CRITICAL for multi-tenancy
    private final String tenantName;
    private final String email;
    private final String password;
    private final Collection<? extends GrantedAuthority> authorities;

    // ... constructors and methods
}

// Usage in a service:
@PreAuthorize("hasRole('USER')")
public List<Order> getMyOrders() {
    TenantAwareUserDetails user = (TenantAwareUserDetails)
        SecurityContextHolder.getContext().getAuthentication().getPrincipal();

    // ALWAYS scope DB queries to the tenant!
    return orderRepository.findByTenantIdAndUserId(user.getTenantId(),
```

```
user.getUserId());
}
```

5.2 Thread-Safe CustomUserDetailsService

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String email)
            throws UsernameNotFoundException {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException(
                "User not found with email: " + email));

        if (!user.isActive()) {
            throw new DisabledException("Account is deactivated");
        }
        if (user.isLocked()) {
            throw new LockedException("Account is locked");
        }

        return new CustomUserDetails(user);
    }
}
```

Performance Consideration: Caching UserDetails

In a JWT-based architecture where your `JwtFilter` calls `loadUserByUsername()` on EVERY request, you are hitting the database on every API call. This defeats the purpose of stateless JWTs!

Solution 1: Extract user info from JWT claims (no DB hit)

```
// In JwtFilter: Build Authentication from JWT claims directly
if (jwtService.isTokenValid(token, username)) {
    List<SimpleGrantedAuthority> authorities =
jwtService.extractRoles(token).stream()
```

```
        .map(SimpleGrantedAuthority::new).toList();

    // Create Authentication WITHOUT calling UserDetailsService!
    UsernamePasswordAuthenticationToken auth =
        new UsernamePasswordAuthenticationToken(username, null, authorities);
    SecurityContextHolder.getContext().setAuthentication(auth);
}
```

Solution 2: Cache UserDetails with Caffeine

```
@Service
public class CachingUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;
    private final Cache<String, UserDetails> cache = Caffeine.newBuilder()
        .maximumSize(10_000)
        .expireAfterWrite(5, TimeUnit.MINUTES)
        .build();

    @Override
    public UserDetails loadUserByUsername(String email) {
        return cache.get(email, this::loadFromDb);
    }

    private UserDetails loadFromDb(String email) {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException("Not found: " +
email));
        return new CustomUserDetails(user);
    }
}
```

Part 6: Password Security and Encoding

6.1 Why Password Encoding is Non-Negotiable

```
Plain text:   password123          (compromised = instant access)
MD5 hash:     482c811da5d5b4...   (weak, rainbow table attacks)
SHA-256:      ef92b778bafe77...   (weak, no salt)
BCrypt:       $2a$10$N9qo8uL...  (strong, salted, slow by design)
```

Deep Concept Explanation

WHAT: Password encoding is the process of converting a plain-text password into a cryptographic hash that CANNOT be reversed. Even if an attacker steals your entire database, they cannot recover the original passwords.

WHY it's non-negotiable: Every major data breach in history (LinkedIn 2012 - 6.5M passwords, Adobe 2013 - 153M passwords) was devastating BECAUSE passwords were stored as plain text or weak hashes (MD5/SHA-1). With BCrypt, even if an attacker steals your user table, cracking each password would take thousands of CPU-years.

WHEN to use which encoder:

- **New project:** Use `BCryptPasswordEncoder(12)` (battle-tested, fast enough for most apps)
- **High-security financial systems:** Use `Argon2PasswordEncoder` (latest standard, resists GPU attacks)
- **Migrating legacy system:** Use `DelegatingPasswordEncoder` (supports both old and new hashes)

Real-World Production Scenario: A Database Leak

```
Scenario: An attacker downloads your entire users table containing 10 million
rows.

With Plain Text Passwords:
  All 10 million accounts are instantly compromised.

With MD5 (no salt):
  The attacker uses a Rainbow Table (pre-computed MD5 hashes of common
```

```
passwords).
  80% of passwords are cracked within minutes.

With SHA-256 (no salt):
  Same attack as MD5 but with SHA-256 rainbow tables.
  Slightly slower but still very fast.

With BCrypt (strength 12):
  Each password verification attempt takes ~200ms.
  To crack ONE password by brute force: 200ms x 1 billion attempts = 6.3 years.
  To crack ALL 10 million: virtually impossible with current hardware.
```

6.2 Password Encoders

Encoder	Strength	Speed	Use In Production?
`BCryptPasswordEncoder`	Strong	~100ms	Yes (Default, highly recommended)
`Argon2PasswordEncoder`	Strongest	~300ms	Yes (Latest industry standard, memory-hard)
`SCryptPasswordEncoder`	Very strong	~200ms	Yes (memory-hard)
`NoOpPasswordEncoder`	None	Instant	NEVER (unit testing only)

How to Choose: Senior Engineer's Decision Framework

```
Question 1: Is this a new greenfield project?
  -> Yes: Use BCryptPasswordEncoder(12). It's the industry standard.

Question 2: Is this a high-security financial/healthcare system?
  -> Yes: Use Argon2PasswordEncoder. It's memory-hard (resists GPU and ASIC
attacks).

Question 3: Am I migrating from a legacy system with SHA/MD5 hashes?
  -> Yes: Use DelegatingPasswordEncoder.
          Register both old (MD5/SHA) and new (BCrypt) encoders.
          Upon login, transparently re-hash from old to new.

Question 4: What BCrypt strength should I use?
  -> Strength 10: ~100ms per hash (good for most web apps)
  -> Strength 12: ~200ms per hash (recommended for sensitive apps)
  -> Strength 14: ~800ms per hash (slows login noticeably, use only if
```

```
justified)
  -> NEVER go below 10 in production!
```

6.3 Advanced Concept: BCrypt Architecture

```java
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12); // Strength 12 (2^12 iterations)
}

// Registration: encode password before saving
String rawPassword = "secret123";
String encoded = passwordEncoder.encode(rawPassword);
// $2a$12$LJ3m4ys1jN6cVKkWJPz2U.Bj6YOyI1L5lq0A9X5JVfKq3O7/HZOW.
// Each encode() call produces DIFFERENT output because the salt is randomly
generated and embedded in the string.
```

How BCrypt Works Internally

```
Input: "secret123"

Step 1: Generate random salt (22 characters)
        Salt: LJ3m4ys1jN6cVKkWJPz2U.

Step 2: Hash password + salt using Blowfish cipher
        Run 2^12 = 4096 iterations (Intentional Work Factor!)

Step 3: Combine: $2a$12$<salt><hash>
        $2a$12$LJ3m4ys1jN6cVKkWJPz2U.Bj6YOyI1L5lq0A9X5JVfKq3O7/HZOW.

Architectural Defense:
1. The random salt prevents Rainbow Table attacks (pre-computed hash
dictionaries).
2. The intentional slowness (4096 iterations) prevents GPU brute-forcing from
easily guessing passwords if your database is leaked.
```

Dissecting a BCrypt Hash String

```
$2a$12$LJ3m4ys1jN6cVKkWJPz2U.Bj6YOyI1L5lq0A9X5JVfKq3O7/HZOW.
 |  |  |_____________________|_______________________________|
 |  |           |                              |
 |  |       22-char salt              31-char hash
 |  |
 | Cost factor (2^12 = 4096 rounds)
 |
 Algorithm version (2a = modern BCrypt)
```

Why `encode()` produces different outputs each time: Each call generates a NEW random 22-character salt. So `encode("secret123")` will produce a different string every time. But `matches("secret123", storedHash)` always works because it extracts the salt FROM the stored hash and re-computes.

6.4 Zero-Downtime Password Migrations: DelegatingPasswordEncoder

If a system used weak MD5 hashes for ten years, how does an Architect securely migrate 1 million users to BCrypt without forcing everyone to reset their passwords?

The answer: **DelegatingPasswordEncoder**.

```java
@Bean
public PasswordEncoder passwordEncoder() {
    Map<String, PasswordEncoder> encoders = new HashMap<>();
    encoders.put("bcrypt", new BCryptPasswordEncoder());
    encoders.put("MD5", new MessageDigestPasswordEncoder("MD5"));

    // Default to BCrypt for all new registrations/updates
    DelegatingPasswordEncoder passwordEncoder =
        new DelegatingPasswordEncoder("bcrypt", encoders);

    // Automatically upgrades MD5 hashes to BCrypt upon successful login!
    return passwordEncoder;
}
```

How it works: The database stores strings like `{MD5}abc123hash...` or `{bcrypt}$2a...`. Spring looks at the `{id}` prefix to know which decoder to use. Upon successful MD5 login, the raw password is automatically re-hashed using BCrypt and saved!

Step-by-Step: Implementing Transparent Password Migration

```java
// Step 1: Configure DelegatingPasswordEncoder (shown above)

// Step 2: Create a custom AuthenticationSuccessHandler that re-hashes on login
@Component
@RequiredArgsConstructor
public class PasswordUpgradeHandler implements AuthenticationSuccessHandler {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request,
            HttpServletResponse response, Authentication authentication) {

        // Check if the stored hash uses the legacy encoder
        CustomUserDetails user = (CustomUserDetails)
authentication.getPrincipal();
        String storedHash = user.getPassword();

        if (storedHash.startsWith("{MD5}") || storedHash.startsWith("{SHA}")) {
            // Re-hash with BCrypt using the raw password from the login form
            String rawPassword = request.getParameter("password");
            String bcryptHash = passwordEncoder.encode(rawPassword);

            userRepository.updatePassword(user.getId(), bcryptHash);
            // Database now stores: {bcrypt}$2a$12$...
            // Next login: Spring uses BCrypt directly, no MD5 involved
        }
    }
}
```

Monolithic vs Microservices: Password Migration Differences

Aspect	Monolithic	Microservices
Migration location	Single `UserDetailsService` handles everything	Only the Auth Service owns user credentials. Other services never see passwords.
Database	One user table in one DB	Auth Service has its own dedicated User DB. Migrating only affects one service.

Rollback strategy	Roll back the JAR deployment	Roll back only the Auth Service container. Other microservices are unaffected.
Testing	Test login flow in one integration test	Test Auth Service in isolation. API Gateway and downstream services don't change.

Part 7: JWT Authentication Architecture (Architect's Deep Dive)

7.1 What is JWT?

JSON Web Token (JWT) is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object.

Deep Concept Explanation

WHAT: JWT is a cryptographically signed JSON document that carries verifiable claims about a user's identity and permissions. Unlike session IDs (which are random strings that reference server-side storage), JWTs are SELF-CONTAINED -- the token itself carries all the information the server needs to authorize the request.

WHY it exists: In traditional session-based authentication, the server stores session data in memory (or Redis). When you scale to 10 servers, all 10 must share session storage (sticky sessions or distributed cache). JWT eliminates this entirely -- the token itself IS the proof of authentication. Any server with the signing key can validate it independently.

WHEN to use JWT:

- REST APIs consumed by SPAs (React/Angular) or mobile apps
- Microservices architectures where multiple services need to verify identity
- Third-party API authentication
- Cross-domain single sign-on (SSO)

WHEN NOT to use JWT:

- Server-rendered MVC apps with Thymeleaf/JSP (sessions are simpler)
- When immediate token revocation is critical (e.g., banking terminals)
- When the payload would be too large (JWTs are sent in EVERY request header)

WHERE it is used in real-world systems:

- **E-commerce platforms:** Mobile app and web SPA both call the same stateless REST APIs
- **Microservices:** API Gateway validates JWT, forwards claims to downstream services
- **IoT systems:** Devices authenticate with long-lived JWTs without needing session management
- **SaaS platforms:** Cross-tenant SSO using JWTs issued by a central auth server

Why JWT Over Session-Based Authentication

Feature	Session-Based	JWT	Architect's Consideration
Storage	Server memory / Redis	Client-side only	JWT saves massive Redis infrastructure costs for high-scale apps.
Scaling	Sticky sessions needed	Any server validates	True statelessness; perfect for horizontal auto-scaling.
Network	Fast (small Cookie)	Slower (Large Header)	JWT size matters. Too many claims = bloated HTTP requests.
Revocation	Instant (delete session)	Difficult (requires blacklist)	JWTs are valid until exp. Revocation requires extra Redis hops.
Cross-Domain	Requires shared cookie domain	Works across ANY domain	JWT excels in cross-origin, multi-service environments
Mobile Apps	Complex cookie management	Simple header attachment	Mobile apps universally prefer JWT

Real-World Production Scenario: Choosing JWT vs Sessions

```
Scenario: An e-commerce company has:
  - A React web SPA running on https://shop.example.com
  - An iOS app
  - An Android app
  - A Spring Boot backend on https://api.example.com

Senior Engineer's Thinking:
  1. "Three different clients (web, iOS, Android) consume the same REST API."
  2. "React SPA is on a different domain -- sessions + cookies require CORS
      complexity and SameSite workarounds."
  3. "Mobile apps don't natively support cookies well -- JWT in Auth header is
simpler."
  4. "We plan to add microservices later (Order Service, Payment Service)."
  5. "Decision: Use JWT. It gives us stateless, cross-platform, scalable auth."
```

Monolithic vs Microservices: JWT Behavior Differences

```
MONOLITHIC APPLICATION:
  Client -> POST /api/auth/login -> Monolith issues JWT (HS256, shared secret)
  Client -> GET /api/orders -> Same Monolith validates JWT

  - Single secret key stored in application.yml
```

```
  - Only one application signs AND verifies tokens
  - Simple, fast, sufficient for most monolithic apps

MICROSERVICES ARCHITECTURE:
  Client -> POST /api/auth/login -> Auth Service issues JWT (RS256, private key)
  Client -> GET /api/orders -> API Gateway validates JWT (public key via JWKS)
                            -> Order Service validates JWT (same public key)

  - Private key is ONLY in the Auth Service (guarded secret)
  - All other services use the Public Key (safe to distribute)
  - JWKS endpoint enables automatic key rotation without deployments
```

7.2 Architect's Dilemma: Symmetric vs Asymmetric Signatures

The most critical architectural decision regarding JWTs is how to secure the signature.

Option A: Symmetric Signing (HS256 - HMAC SHA-256)

- **Mechanics:** The **same secret key** is used to both *sign* the token (Auth Server) and *verify* the token (Resource Server/API Gateway).
- **Pros:** Fast, simple to implement (`jjwt-api`).
- **Cons (The Architect's nightmare):** Every microservice that needs to verify the token MUST have a copy of the secret key. If one downstream microservice is compromised, the secret key leaks, allowing the attacker to mint fake tokens and compromise the entire cluster.

Option B: Asymmetric Signing (RS256 - RSA Signature)

- **Mechanics:** Uses a Public/Private key pair. The **Private Key** signs the token (Auth Server only). The **Public Key** verifies it (All other microservices).
- **Pros:** Downstream services only hold the Public Key. Even if hacked, they cannot create fake tokens!
- **Cons:** Slower verification, requires PKI/Key management.

The Enterprise Solution: JWKS (JSON Web Key Set)

In a modern microservices architecture (like Keycloak + Spring Boot), the Auth server exposes its Public Keys via a standard HTTP endpoint (e.g., `/.well-known/jwks.json`). Downstream Spring Boot REST APIs cache these keys and verify RS256 JWTs automatically without any shared secrets.

If a key is compromised, the Auth Server generates a new Private/Public key pair (Key Rotation) and publishes the new Public Key to the JWKS endpoint. Spring Security automatically fetches the new key!

Internal Working: How JWKS Key Rotation Works

```
Timeline of Zero-Downtime Key Rotation:

Day 0: Auth Server signs all JWTs with Key Pair A (kid="key-A")
       JWKS endpoint returns: [PublicKey-A]
       All microservices cache PublicKey-A

Day 30: Security team decides to rotate keys
        Auth Server generates Key Pair B

Day 30: Auth Server starts signing NEW tokens with Key Pair B (kid="key-B")
        JWKS endpoint now returns: [PublicKey-A, PublicKey-B]  <- BOTH keys!

Day 30-31: Some tokens in-flight still use kid="key-A"
           Some new tokens use kid="key-B"
           Microservices see kid="key-B", don't have it cached
           -> Fetch updated JWKS -> Get PublicKey-B -> Cache it
           -> Continue validating both old and new tokens

Day 31: All key-A tokens have expired (15 min access tokens)
        Auth Server removes PublicKey-A from JWKS
        JWKS endpoint returns: [PublicKey-B]

Result: Zero downtime. No deployments. No config changes in any microservice.
```

Real-World Production Scenario: Securing File Upload API with JWT

```
Scenario: Users upload profile pictures to POST /api/files/upload

Senior Engineer's Thinking:
  1. "The upload endpoint must be authenticated -- only logged-in users can
upload."
  2. "I need the userId from the JWT to associate the file with the correct
user."
  3. "Max file size is 5MB. I should validate this BEFORE processing."
  4. "The JWT must have the 'file:write' authority/scope."
  5. "After upload, store the file in S3 and save the URL in the DB."

Implementation:
  @PostMapping("/upload")
  @PreAuthorize("hasAuthority('file:write')")
  public ResponseEntity<FileResponse> upload(@RequestParam MultipartFile file) {
      // Extract userId from SecurityContext (populated by JwtFilter)
      Long userId = ((CustomUserDetails) SecurityContextHolder.getContext()
          .getAuthentication().getPrincipal()).getId();
```

```
    String url = fileService.uploadToS3(file, userId);
    return ResponseEntity.ok(new FileResponse(url));
}
```

7.3 JWT Structure & Claims

```
Header.Payload.Signature

eyJhbGciOiJSUzI1NiIsImtpZCI6IjEyMyJ9.eyJzdWIiOiJ1c2VyQG1haWwuY29tIn0.Sig...
```

Header: Contains Algorithm (`RS256`) and Key ID (`kid` - tells the server which public key from the JWKS to use). **Payload (Claims):**

- `sub` (Subject): the user identifier.
- `iat` (Issued At): epoch time.
- `exp` (Expiration): epoch time.
- `iss` (Issuer): URL of the Auth Server. Prevent tokens from *Dev* being used in *Prod*.
- `aud` (Audience): Who is the token intended for? Prevents Token Substitution attacks.

Deep Dive: Custom Claims and Token Size Impact

Standard (Registered) Claims are defined in RFC 7519:

```
{
  "sub": "user@mail.com",
  "iss": "https://auth.mycompany.com",
  "aud": "https://api.mycompany.com",
  "iat": 1710144000,
  "exp": 1710144900,
  "jti": "a1b2c3d4-unique-token-id"
}
```

Custom (Private) Claims carry application-specific data:

```
{
  "sub": "user@mail.com",
  "roles": ["ROLE_ADMIN", "ROLE_MANAGER"],
```

```
  "tenantId": 42,
  "permissions": ["order:read", "order:write", "user:delete"],
  "department": "Engineering"
}
```

Architect's Warning: Token Bloat Every claim increases the JWT size. The JWT is sent as an HTTP header on EVERY request. If your token has 50 custom claims, it could be 4KB+ per request. For high-throughput APIs handling 10,000 requests/second, that's 40MB/s of extra network overhead just for tokens.

Rule of Thumb: Keep JWTs under 1KB. Include only: `sub`, `roles`, `tenantId`, `exp`, `iss`, `jti`. Fetch additional user details from the database or a User Profile API when needed.

Security Pitfalls: JWT Claims

Mistake	Risk	Fix
Not validating `iss` (Issuer)	Dev environment tokens work in Production	Always verify `iss` matches your Production Auth Server URL
Not validating `aud` (Audience)	Token issued for Service A works on Service B	Each service should check `aud` matches its own identifier
Storing sensitive data in JWT payload	Anyone can decode a JWT (it's Base64, NOT encrypted)	Never put passwords, SSN, credit cards in JWT claims
Not setting `jti` (JWT ID)	Cannot revoke specific tokens or detect replay	Always include `jti` for blacklisting and idempotency
Not validating `exp` before processing	Expired tokens accepted by buggy custom validation	Use the library's built-in expiration check (jjwt, nimbus)

7.4 JWT Service Implementation (Symmetric Example)

```java
@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.access-token-expiration}")
    private long accessTokenExpiry;  // 15 minutes max!
```

```
    @Value("${jwt.refresh-token-expiration}")
    private long refreshTokenExpiry;  // 7 days

    public String generateAccessToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("roles", userDetails.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority).toList());

        return Jwts.builder()
            .setClaims(claims)
            .setSubject(userDetails.getUsername())
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() +
accessTokenExpiry))
            .signWith(getSigningKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) &&
!isTokenExpired(token);
    }

    // Architect Note: The parsing step validates the cryptographic signature.
    // If an attacker altered the payload, `parseClaimsJws` will throw an
Exception!
    private Claims extractAllClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
    }
}
```

Step-by-Step: Complete JwtAuthenticationFilter Implementation

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;
```

```java
    @Override
    protected void doFilterInternal(HttpServletRequest request,
            HttpServletResponse response, FilterChain filterChain)
            throws ServletException, IOException {

        // Step 1: Extract the Authorization header
        final String authHeader = request.getHeader("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            // No JWT present; continue chain (public endpoint or will fail at
AuthorizationFilter)
            filterChain.doFilter(request, response);
            return;
        }

        // Step 2: Extract the token (remove "Bearer " prefix)
        final String jwt = authHeader.substring(7);

        // Step 3: Extract username from token claims
        final String username = jwtService.extractUsername(jwt);

        // Step 4: Only authenticate if not already authenticated (avoid double
processing)
        if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {

            // Step 5: Load user from database (or cache)
            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

            // Step 6: Validate token (signature + expiration + username match)
            if (jwtService.isTokenValid(jwt, userDetails)) {

                // Step 7: Create authenticated token with authorities
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());

                // Step 8: Attach request details (IP, session info)
                authToken.setDetails(
                    new
WebAuthenticationDetailsSource().buildDetails(request));

                // Step 9: Set authentication in SecurityContext
                // Now AuthorizationFilter will see this user as authenticated!

SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
```

```
        // Step 10: Continue the filter chain
        filterChain.doFilter(request, response);
    }

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        // Performance optimization: skip JWT filter for public endpoints
        String path = request.getServletPath();
        return path.startsWith("/api/auth/") ||
path.startsWith("/api/public/");
    }
}
```

Real-World Production Scenario: Refresh Token Flow

```
Complete Token Lifecycle in Production:

1. User logs in:
   POST /api/auth/login -> Returns: { accessToken (15 min), refreshToken (7
days) }

2. Client stores both tokens:
   - accessToken: in memory (JavaScript variable) -- NEVER in localStorage!
   - refreshToken: in HttpOnly Secure cookie -- invisible to JavaScript (XSS-
proof)

3. Client makes API calls:
   GET /api/orders
   Authorization: Bearer <accessToken>

4. After 15 minutes, accessToken expires:
   GET /api/orders -> 401 Unauthorized (token expired)

5. Client silently refreshes:
   POST /api/auth/refresh
   Cookie: refreshToken=<refreshToken>
   -> Auth Server validates the refresh token
   -> Issues a NEW accessToken (+ optionally rotates refresh token)
   -> Returns: { accessToken (15 min), refreshToken (7 days - NEW) }

6. Client retries the original request with the new accessToken.
   GET /api/orders -> 200 OK

7. When user clicks "Logout":
```

```
  POST /api/auth/logout
  -> Server blacklists BOTH tokens in Redis
  -> Clears the refreshToken cookie
```

```java
// Refresh Token Endpoint
@PostMapping("/refresh")
public ResponseEntity<AuthResponse> refresh(
        @CookieValue(name = "refreshToken") String refreshToken) {

    // Validate the refresh token
    if (!jwtService.isTokenValid(refreshToken)) {
        throw new BadCredentialsException("Invalid or expired refresh token");
    }

    // Check if refresh token is blacklisted (user logged out)
    String jti = jwtService.extractJti(refreshToken);
    if (redisTemplate.hasKey("blacklist:" + jti)) {
        throw new BadCredentialsException("Refresh token has been revoked");
    }

    // Issue new tokens
    String username = jwtService.extractUsername(refreshToken);
    UserDetails user = userDetailsService.loadUserByUsername(username);
    String newAccessToken = jwtService.generateAccessToken(user);
    String newRefreshToken = jwtService.generateRefreshToken(user);

    // Rotate: blacklist the OLD refresh token
    long remainingTtl = jwtService.extractExpiration(refreshToken).getTime()
                        - System.currentTimeMillis();
    redisTemplate.opsForValue().set("blacklist:" + jti, "revoked",
        remainingTtl, TimeUnit.MILLISECONDS);

    // Set new refresh token as HttpOnly cookie
    ResponseCookie cookie = ResponseCookie.from("refreshToken",
newRefreshToken)
        .httpOnly(true).secure(true).sameSite("Strict")
        .path("/api/auth").maxAge(Duration.ofDays(7)).build();

    return ResponseEntity.ok()
        .header(HttpHeaders.SET_COOKIE, cookie.toString())
        .body(new AuthResponse(newAccessToken));
}
```

Part 8: Spring Security with OAuth2 & OIDC

The Grand Misconception: Identity vs Authorization

Many developers confuse **OAuth2** with an Authentication Protocol. It is NOT.

- **OAuth2 is a delegated AUTHORIZATION protocol.** (The user gives an App permission to access their Google Calendar via Scopes).
- **OpenID Connect (OIDC)** is an IDENTITY protocol built on top of OAuth2. It adds an **ID Token** (a JWT) alongside the OAuth2 Access Token. (The user proves WHO they are to the App).

Deep Concept Explanation: OAuth2 vs OIDC Internals

WHAT:

- **OAuth2** answers: "Is this Application allowed to access this User's data?" (Delegated AUTHORIZATION)
- **OIDC** answers: "WHO is this User?" (AUTHENTICATION/IDENTITY)

WHY this distinction matters: A pure OAuth2 Access Token tells you *nothing* about the user's identity. It only says: "This application is allowed to read the user's email." To know WHO the user is, you need OIDC's `ID Token` -- a JWT containing `sub` (user ID), `name`, `email`, `picture`, etc.

Real-World Confusion Example:

```
Scenario: Your Spring Boot app uses "Login with Google" button.

WRONG approach (OAuth2 only):
  1. User clicks "Login with Google"
  2. You receive an Access Token
  3. You call Google's API: GET https://www.googleapis.com/oauth2/v2/userinfo
  4. You get the user's profile -- but you made an extra API call!

CORRECT approach (OIDC):
  1. User clicks "Login with Google"
  2. You receive BOTH an Access Token AND an ID Token (JWT)
  3. The ID Token ALREADY contains sub, email, name, picture
  4. No extra API call needed! Parse the ID Token locally.
```

WHEN to use OAuth2 vs OIDC:

Scenario	Use
"Login with Google/GitHub"	OIDC (you need to know WHO the user is)

"Allow Slack to post to my channel"	OAuth2 (Slack needs permission, not identity)
"SSO across microservices with Keycloak"	OIDC (Keycloak returns ID tokens)
"GitHub Actions accessing AWS resources"	OAuth2 Client Credentials (machine-to-machine)

8.1 OAuth2 / OIDC Roles

1. **Resource Owner:** The human user.
2. **Client Application:** Your Spring Boot Application.
3. **Authorization Server:** Keycloak, Auth0, Google, GitHub.
4. **Resource Server:** Your internal REST API.

Internal Working: Complete Authorization Code Flow

```
Step 1: User clicks "Login with Google" on your React SPA
        Browser redirects to:
        https://accounts.google.com/authorize?
          client_id=YOUR_APP_ID
          &redirect_uri=https://yourapp.com/callback
          &response_type=code
          &scope=openid profile email
          &state=random_csrf_string

Step 2: User sees Google's consent screen
        "YourApp wants to access your email and profile"
        User clicks "Allow"

Step 3: Google redirects back to YOUR app with an Authorization Code
        https://yourapp.com/callback?code=AUTH_CODE_123&state=random_csrf_string

Step 4: Your Spring Boot backend exchanges the code for tokens
        POST https://oauth2.googleapis.com/token
        Body: {
          code: AUTH_CODE_123,
          client_id: YOUR_APP_ID,
          client_secret: YOUR_SECRET,    <- Server-to-server, never exposed!
          redirect_uri: https://yourapp.com/callback,
          grant_type: authorization_code
        }

Step 5: Google returns tokens
        {
          access_token: "ya29...",          <- For calling Google APIs
          id_token: "eyJhbGci...",          <- JWT with user identity (OIDC)
          refresh_token: "1//04dX...",      <- For getting new access tokens
```

```
          token_type: "Bearer",
          expires_in: 3600
        }

Step 6: Your app parses the ID Token to get user info
        { sub: "118234123", email: "user@gmail.com", name: "John" }

Step 7: Your app creates/updates a local user record
        Generates YOUR app's own JWT for subsequent API calls
```

8.2 Scope vs Role (Architectural Clarity)

In an enterprise deployment, do not confuse `Roles` with `Scopes`.

- **Role (RBAC):** `ROLE_ADMIN`, `ROLE_MANAGER`. Defines what the User is allowed to do within the system. Checked via `@PreAuthorize("hasRole('ADMIN')")`.
- **Scope (OAuth2):** `read:profile`, `write:orders`. Defines what the *Client Application* is allowed to do ON BEHALF OF the User. Checked via `@PreAuthorize("hasAuthority('SCOPE_read:profile')")`.

If a User is an `ADMIN` but logs in via a third-party App that only requested `read:profile` scope, the token should be rejected for `DELETE` operations, despite the User's inherent admin role. The *App* doesn't have the scope!

Real-World Production Scenario: Combining Scopes and Roles

```
Scenario: Enterprise CRM with a public API for third-party integrations.

Internal Admin Dashboard (Thymeleaf/React):
  - User logs in directly via Keycloak OIDC
  - Token contains roles: [ROLE_ADMIN]
  - Token contains scopes: [openid, profile, ALL permissions]
  - Admin can delete customers: @PreAuthorize("hasRole('ADMIN')")

Third-party Zapier Integration:
  - Zapier authenticates via OAuth2 Authorization Code Flow
  - User grants Zapier only the "read:contacts" scope
  - Token contains roles: [ROLE_ADMIN]  (user IS an admin)
  - Token contains scopes: [read:contacts]  (app only has read access)
  - Zapier tries to DELETE a customer -> REJECTED!
    @PreAuthorize("hasRole('ADMIN') AND hasAuthority('SCOPE_write:contacts')")
    -> User has ROLE_ADMIN ✓ but app doesn't have SCOPE_write:contacts ✗
```

8.3 OAuth2 Resource Server Configuration (The Modern Way)

When using Keycloak or Auth0, your Spring Boot Application acts as a `Resource Server`. It does NOT mint tokens. It only verifies them using JWKS!

```yaml
# application.yml -> No secrets required! Only the JWKS URL.
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.mycompany.com/realms/enterprise
          jwk-set-uri:
https://auth.mycompany.com/realms/enterprise/protocol/openid-connect/certs
```

```java
@Configuration
@EnableWebSecurity
public class OAuth2ResourceServerConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
        return http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .anyRequest().authenticated()
            )
            // Spring magically handles RS256 token verification via JWKS!
            .oauth2ResourceServer(oauth2 -> oauth2.jwt())
            .build();
    }
}
```

Step-by-Step: Custom JWT Claims Mapping from Keycloak

By default, Keycloak stores roles inside `realm_access.roles` in the JWT. Spring Security doesn't read that path automatically. You need a custom `JwtAuthenticationConverter`:

```java
@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter grantedAuthoritiesConverter =
        new JwtGrantedAuthoritiesConverter();
    grantedAuthoritiesConverter.setAuthoritiesClaimName("realm_access.roles");
    grantedAuthoritiesConverter.setAuthorityPrefix("ROLE_");

    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(jwt -> {
        // Extract roles from Keycloak's nested JWT structure
        Map<String, Object> realmAccess = jwt.getClaimAsMap("realm_access");
        if (realmAccess == null) return Collections.emptyList();

        List<String> roles = (List<String>) realmAccess.get("roles");
        return roles.stream()
            .map(role -> new SimpleGrantedAuthority("ROLE_" +
role.toUpperCase()))
            .collect(Collectors.toList());
    });
    return converter;
}

// Usage in SecurityFilterChain:
.oauth2ResourceServer(oauth2 -> oauth2
    .jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthenticationConverter())))
```

Part 9: Role-Based Access Control (RBAC)

What is RBAC?

Role-Based Access Control assigns permissions through **roles** rather than directly to users.

Deep Concept Explanation

WHAT: RBAC is an access control model where users are assigned to roles, and roles are assigned permissions. The user never interacts with permissions directly -- they inherit them through their role assignment.

WHY RBAC is the enterprise standard: Without RBAC, managing permissions for 10,000 users requires individually assigning each permission to each user. With RBAC, you assign a user one role (e.g., `MANAGER`), and all 15 permissions associated with that role are automatically inherited. When a new permission is added to the role, all 500 managers get it instantly.

WHERE RBAC is used in real-world systems:

- **Hospital systems:** `DOCTOR` can view all patient records, `NURSE` can view assigned patients only, `RECEPTIONIST` can view appointment schedules
- **E-commerce:** `CUSTOMER` can place orders, `VENDOR` can manage inventory, `ADMIN` can manage everything
- **Financial systems:** `TELLER` can process transactions under $10K, `MANAGER` can approve transactions over $10K

Design Pattern: Two-Level Authorization

Spring Security supports both coarse-grained (role) and fine-grained (authority) access control:

- **URL-based security** (coarse) -- configured in `SecurityFilterChain`
- **Method-level security** (fine) -- `@PreAuthorize`, `@Secured` annotations

9.1 Roles vs Authorities

```
ROLE: A grouping of permissions (coarse-grained)
  - ROLE_USER, ROLE_ADMIN

AUTHORITY / PERMISSION: A specific action (fine-grained)
  - order:read, order:write, user:delete
```

```
Hierarchy Strategy (Architect's Approach):
  ROLE_ADMIN -> [user:read, user:delete, order:read, order:write, report:export]
  ROLE_MANAGER -> [user:read, order:read, order:write, report:export]
  ROLE_USER -> [order:read]
```

Internal Working: How Spring Security Stores Roles vs Authorities

Both `roles` and `authorities` are stored as `GrantedAuthority` objects inside the `Authentication` object. Spring appends `ROLE_` prefix automatically when you use `hasRole()` :

```
// These two are EQUIVALENT:
.hasRole("ADMIN")           // Spring internally checks for "ROLE_ADMIN"
.hasAuthority("ROLE_ADMIN") // Direct authority check (no prefix magic)

// These are DIFFERENT:
.hasRole("ADMIN")           // Looks for authority string "ROLE_ADMIN"
.hasAuthority("ADMIN")      // Looks for authority string "ADMIN" (without
ROLE_ prefix)
```

Real-World Production Scenario: Database-Driven RBAC

```
// JPA Entity Model for dynamic RBAC
@Entity
public class UserEntity {
    @Id @GeneratedValue
    private Long id;
    private String email;
    private String passwordHash;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles")
    private Set<RoleEntity> roles;
}

@Entity
public class RoleEntity {
    @Id @GeneratedValue
    private Long id;
    private String name; // ADMIN, MANAGER, USER
```

```java
    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "role_permissions")
    private Set<PermissionEntity> permissions;
}

@Entity
public class PermissionEntity {
    @Id @GeneratedValue
    private Long id;
    private String name; // order:read, order:write, user:delete
}

// In CustomUserDetails constructor:
public CustomUserDetails(UserEntity user) {
    this.authorities = new ArrayList<>();
    for (RoleEntity role : user.getRoles()) {
        // Add the role itself
        authorities.add(new SimpleGrantedAuthority("ROLE_" + role.getName()));
        // Add all permissions belonging to this role
        for (PermissionEntity perm : role.getPermissions()) {
            authorities.add(new SimpleGrantedAuthority(perm.getName()));
        }
    }
}
// Result: authorities = [ROLE_ADMIN, order:read, order:write, user:delete,
...]
```

9.2 URL-Based Security

```java
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            // Public endpoints
            .requestMatchers("/api/auth/**").permitAll()

            // Coarse Role-based
            .requestMatchers("/api/admin/**").hasRole("ADMIN")

            // Fine-grained Authority-based. Even if you are an ADMIN,
            // you must explicitly have the order:write authority mapped.
            .requestMatchers(HttpMethod.POST, "/api/orders/**")
                .hasAuthority("order:write")
```

```
            .anyRequest().authenticated()
        )
        .build();
}
```

Security Pitfall: Ordering of URL Matchers

URL matchers are evaluated in ORDER (top-to-bottom). The FIRST match wins. This is a very common source of bugs:

```
// WRONG ORDER: Everything matches the first rule!
.authorizeHttpRequests(auth -> auth
    .anyRequest().authenticated()           // <- Matches EVERYTHING first!
    .requestMatchers("/api/public/**").permitAll()  // <- NEVER reached!
)

// CORRECT ORDER: Specific rules first, catch-all last
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/public/**").permitAll()   // <- Specific first
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated()                     // <- Catch-all last
)
```

9.3 Method-Level Security (AOP Based)

Method security utilizes Aspect-Oriented Programming (AOP). Spring proxies your beans and checks permissions *before* (or *after*) executing the actual code.

```
@Configuration
@EnableMethodSecurity  // Replaces deprecated @EnableGlobalMethodSecurity
public class MethodSecurityConfig { }

@Service
public class UserService {

    // Checked BEFORE method execution via SpEL (Spring Expression Language)
    @PreAuthorize("hasRole('ADMIN')")
    public void deleteUser(Long userId) {
        userRepository.deleteById(userId);
    }
```

```
    // Advanced: Business Logic in Security! (ABAC - Attribute Based Access
Control)
    // Only an ADMIN, OR the user requesting their own profile can execute
this.
    @PreAuthorize("hasRole('ADMIN') or #userId == authentication.principal.id")
    public User getUser(@Param("userId") Long userId) {
        return userRepository.findById(userId).orElseThrow();
    }
}
```

Deep Concept Explanation: How AOP-Based Security Works Internally

```
1. Spring Boot starts and scans @EnableMethodSecurity
2. Spring creates a BeanPostProcessor that intercepts bean creation
3. For every @Service/@Component with @PreAuthorize annotations:
   - Spring creates a PROXY (either JDK Dynamic Proxy or CGLIB subclass)
   - The proxy wraps the original bean
4. When code calls userService.deleteUser(42):
   - The call hits the PROXY first (not the real method)
   - The proxy evaluates the SpEL expression: hasRole('ADMIN')
   - If the current Authentication.getAuthorities() contains ROLE_ADMIN ->
PROCEED
   - If not -> throw AccessDeniedException (403)
5. After @PreAuthorize passes, the REAL method executes

IMPORTANT GOTCHA: If you call deleteUser() from within the SAME class,
the proxy is BYPASSED! This is a well-known Spring AOP limitation.

// THIS WILL NOT CHECK SECURITY:
@Service
public class UserService {
    public void someMethod() {
        this.deleteUser(42);  // Direct internal call = proxy is bypassed!
    }

    @PreAuthorize("hasRole('ADMIN')")
    public void deleteUser(Long id) { ... }
}

// FIX: Inject the bean into itself or use AopContext
```

Real-World Production Scenario: Multi-Tenant Data Isolation

```java
// Ensuring users can ONLY access data belonging to their tenant
@Service
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepository orderRepository;

    @PreAuthorize("hasRole('USER')")
    public List<Order> getOrders() {
        // Extract tenant from SecurityContext
        TenantAwareUserDetails user = (TenantAwareUserDetails)

SecurityContextHolder.getContext().getAuthentication().getPrincipal();

        // CRITICAL: Always filter by tenantId!
        // Without this, User A could see User B's orders from a different
company!
        return orderRepository.findByTenantId(user.getTenantId());
    }

    @PostAuthorize("returnObject.tenantId ==
authentication.principal.tenantId")
    public Order getOrderById(Long orderId) {
        // @PostAuthorize runs AFTER the method returns
        // If the returned Order belongs to a different tenant -> 403 Forbidden
        return orderRepository.findById(orderId).orElseThrow();
    }
}
```

Part 10: Security Filter Chain Deep Dive & Internal Mechanics

Why Understanding the Filter Chain Matters

- To add **custom authentication** (JWT filter), you must know WHERE to insert it.
- To debug security issues, you need to understand WHICH filter is rejecting the request.

10.1 Default Filter Execution Order

```
HTTP Request
     |
     v
+-------------------------------------------------------+
| 1. SecurityContextPersistenceFilter                    |
|    Load SecurityContext from JVM ThreadLocal            |
|                    |                                    |
| 2. HeaderWriterFilter                                  |
|    Add security headers (Strict-Transport-Security)    |
|                    |                                    |
| 3. CsrfFilter                                         |
|    Validate CSRF token for state-changing endpoints    |
|                    |                                    |
| 4. LogoutFilter                                        |
|    Intercepts POST /logout and invalidates session     |
|                    |                                    |
| 5. UsernamePasswordAuthenticationFilter                |
|    (Your custom JWT Filter goes BEFORE this one usually)|
|                    |                                    |
| 6. BearerTokenAuthenticationFilter                     |
|    OAuth2 Resource Server JWT Validator                |
|                    |                                    |
| 11. SessionManagementFilter                            |
|     Verifies concurrency, prevents session fixation    |
|                    |                                    |
| 12. ExceptionTranslationFilter                         |
|     CATCHES EXCEPTIONS thrown heavily down the chain   |
|     AuthnException -> 401, AccessDeniedException -> 403|
|                    |                                    |
| 13. AuthorizationFilter                                |
|     Evaluates URL matchers defined in authorizeHttpReqs|
+-------------------------------------------------------+
     |
```

```
    v (if authorized)
Controller handles request
```

Deep Concept Explanation: Why Filter Ordering is Critical

WHERE your custom JWT filter must be placed:

```
// Your JwtFilter BEFORE UsernamePasswordAuthenticationFilter
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
```

WHY `addFilterBefore(UsernamePasswordAuthenticationFilter.class)`?

Because `UsernamePasswordAuthenticationFilter` only handles form-login POST requests to `/login`. Your JWT filter needs to run BEFORE it so that JWT-based requests are authenticated first. If your JWT filter runs AFTER the authorization filter, requests hit the authorization check with `principal = null` and get immediately rejected with 403.

Real-World Debugging: Common Filter Chain Issues

```
Problem 1: "My JwtFilter runs but the request still returns 401"
  Debug: Is the SecurityContext cleared between requests?
  Check: Are you using SessionCreationPolicy.STATELESS?
  If not, Spring may try to load a session that doesn't exist.

Problem 2: "CORS preflight OPTIONS request returns 403"
  Debug: The OPTIONS request has NO Authorization header
  Fix: Ensure CorsFilter runs BEFORE the security chain
  http.cors(cors -> cors.configurationSource(...))
  AND: .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()

Problem 3: "My custom filter runs TWICE per request"
  Debug: You registered the filter both as a @Component (auto-scanned)
         AND via addFilterBefore() in the SecurityConfig
  Fix: Either remove @Component from the filter OR use a
       FilterRegistrationBean to disable auto-registration
```

```
// Fix for double-registration of custom filters
@Bean
public FilterRegistrationBean<JwtAuthenticationFilter> jwtFilterRegistration(
        JwtAuthenticationFilter filter) {
    FilterRegistrationBean<JwtAuthenticationFilter> registration =
```

```
        new FilterRegistrationBean<>(filter);
    registration.setEnabled(false); // Prevent Servlet container from auto-registering
    return registration;
}
```

10.2 ExceptionTranslationFilter Explained

Why do you see a default Spring Boot login page when you browse to a secured URL without being authenticated?

The `AuthorizationFilter` (#13) sees you aren't authenticated and throws an `AccessDeniedException`. The `ExceptionTranslationFilter` (#12) catches it! It realizes you aren't authenticated, so it triggers the `AuthenticationEntryPoint`, which by default returns an `HTTP 302 Redirect to /login`!

If you are building a REST API, you don't want an HTML page returned. You must override the entry points!

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .exceptionHandling(ex -> ex
            .authenticationEntryPoint((request, response, exception) -> {
                response.setStatus(401); // Halt the redirect! Return JSON!
                response.setContentType("application/json");
                response.getWriter().write("{\"error\": \"Authentication Required\"}");
            })
            .accessDeniedHandler((request, response, exception) -> {
                response.setStatus(403);
                response.setContentType("application/json");
                response.getWriter().write("{\"error\": \"Insufficient Privileges\"}");
            })
        )
        .build();
}
```

Real-World Production Scenario: Structured Error Responses

In production, you want structured error responses with error codes, timestamps, and request paths:

```
@Component
public class CustomAuthenticationEntryPoint implements AuthenticationEntryPoint
{

    private final ObjectMapper objectMapper = new ObjectMapper();

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse
response,
            AuthenticationException exception) throws IOException {

        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);

        Map<String, Object> errorBody = Map.of(
            "timestamp", Instant.now().toString(),
            "status", 401,
            "error", "Unauthorized",
            "message", "Authentication is required to access this resource",
            "path", request.getRequestURI()
        );

        objectMapper.writeValue(response.getOutputStream(), errorBody);
    }
}
```

Part 11: Session-Based vs Token-Based Authentication

11.1 Advanced Session Management (Session Registry)

In monolithic (Session-based) architectures, how do you prevent a user from logging in on 5 different devices at once?

Spring Security utilizes the `SessionRegistry` to track active sessions across the JVM.

Deep Concept Explanation

WHAT: Session management controls HOW the server maintains user state between HTTP requests. In session-based auth, the server stores user data in memory (or Redis) and sends a `JSESSIONID` cookie to the browser. The browser sends this cookie on every subsequent request.

WHY sessions still exist in the JWT era: Not every application needs stateless JWT. Server-rendered MVC apps (Thymeleaf, JSP), admin dashboards, and internal tools are simpler and more secure with sessions because:

- Instant revocation (delete the session = user is immediately logged out)
- No token size concerns (the cookie is tiny)
- No client-side token storage risks (HttpOnly cookies are immune to XSS)

WHEN to use sessions vs tokens:

Criteria	Use Sessions	Use JWT
App type	Server-rendered (Thymeleaf/JSP)	REST API + SPA/Mobile
Scaling	Single server or Redis-backed sessions	Horizontally scaled, stateless
Revocation	Immediate logout is critical	Short-lived tokens are acceptable
Clients	Browser only	Browser + Mobile + Third-party

```
@Bean
public SecurityFilterChain sessionConfig(HttpSecurity http) throws Exception {
    return http
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
            // Session Fixation Defense: Generate a brand NEW JSESSIONID upon
login
```

```
            .sessionFixation().migrateSession()
            // Concurrency Control: Only 1 active login allowed across all
browsers
            .maximumSessions(1)
            // If true, the second login attempt throws an error.
            // If false, the first login is aggressively invalidated.
            .maxSessionsPreventsLogin(true)
            .expiredUrl("/login?expired=true")
        )
        .build();
}
```

Architect's Note: If you scale this app to 3 EC2 instances, the `SessionRegistry` exists in local memory. Concurrency control will BREAK unless you implement Spring Session distributed backed by Redis (`@EnableRedisHttpSession`).

Monolithic vs Microservices: Session Management

```
MONOLITHIC (Session-Based):
  Client -> POST /login -> Server creates JSESSIONID, stores in memory/Redis
  Client -> GET /dashboard -> Server reads JSESSIONID cookie, loads session

  Scaling:
  - 1 server: Works perfectly with in-memory sessions
  - 3 servers behind Load Balancer:
    Option A: Sticky sessions (LB routes same user to same server) -- fragile
    Option B: Spring Session + Redis (sessions stored centrally) -- recommended
    Option C: Switch to JWT (eliminate sessions entirely) -- best for new
projects

MICROSERVICES (Token-Based):
  Client -> POST /auth/login -> Auth Service generates JWT
  Client -> GET /api/orders -> Order Service validates JWT (no session needed)
  Client -> GET /api/profile -> User Service validates JWT (no session needed)

  Each service is independently stateless.
  No shared session storage required.
  Each service caches JWKS keys for JWT validation.
```

11.2 Stateless (Token-Based) Configuration

```java
@Bean
public SecurityFilterChain tokenConfig(HttpSecurity http) throws Exception {
    return http
        // Token Architectures are immune to CSRF if implemented properly via
Headers
        .csrf(csrf -> csrf.disable())
        .sessionManagement(session ->
            // Prevents Spring Security from creating a JSESSIONID entirely
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        // Must inject custom JWT reader
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

Real-World Production Scenario: Handling Token Expiration Gracefully

```
Scenario: A user is filling out a long form (30 min). Their access token
          expires after 15 min. When they submit the form -> 401 error!

Senior Engineer's Solution:
  1. Frontend interceptor (Axios/Fetch) catches 401 responses
  2. BEFORE showing an error, attempt a silent token refresh:
     - Call POST /api/auth/refresh with the refresh token cookie
     - If successful: retry the original request with the new access token
     - If refresh fails (refresh token expired too): redirect to login page

This is called "Silent Refresh" or "Token Rotation."
The user NEVER sees the 401. The form submission succeeds transparently.
```

Interview Questions: Session vs Token Architecture

Q: If your JWT-based REST API accidentally creates a JSESSIONID cookie, what went wrong?
Answer: You forgot to set `SessionCreationPolicy.STATELESS`. Without this, Spring Security creates a session by default after authentication. The fix is adding `.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))`.

Q: Can you use BOTH session-based and token-based authentication in the same application?
Answer: Yes! Configure two separate `SecurityFilterChain` beans with different `securityMatcher()` patterns. The admin dashboard chain uses form-login with sessions. The API chain uses JWT with stateless sessions. Spring matches the incoming URL to the correct chain.

Part 12: Microservices Security Architecture (Zero-Trust)

12.1 Architect's View: Zero-Trust Security

Historically, IT organizations used **Perimeter Security** (the "Castle and Moat" architecture). If a request originated from outside the VPN, it was untrusted. If a request originated from *inside* the VPN (e.g., between two internal microservices), it was implicitly trusted.

Zero-Trust Architecture dictates: "Never trust, always verify." Every single network request--even between two internal microservices strictly inside AWS VPC--must be authenticated, authorized, and encrypted.

Deep Concept Explanation

WHAT: Zero-Trust is a security model where NO entity (user, service, device, network segment) is automatically trusted, regardless of its physical or network location. Every access request must be authenticated, authorized, and encrypted before being granted.

WHY it exists: The Perimeter Security model failed catastrophically in the era of cloud, microservices, and remote work. If an attacker breaches a single microservice inside a VPC (e.g., via a Log4j vulnerability), Perimeter Security gives them free, unrestricted access to EVERY other internal service -- because internal traffic is "trusted." Zero-Trust prevents this lateral movement.

WHEN to adopt Zero-Trust:

- When running microservices on Kubernetes/ECS (services communicate over internal networks)
- When multiple teams own different services (a compromised team's service shouldn't affect others)
- When handling sensitive data (healthcare, financial, government)
- When regulatory compliance requires it (PCI-DSS, HIPAA, SOC2)

WHERE it is used in real-world systems:

- **Google's BeyondCorp:** Every Google employee's laptop is treated as if it's on the public internet. No VPN needed; every request is individually authenticated.
- **Netflix:** Every microservice-to-microservice call is authenticated using mTLS via their Zuul gateway.
- **Banks:** Internal trading APIs are protected with certificate-based authentication even between internal services.

Two Patterns for Internal Security: API Gateway vs Service Mesh

Pattern	API Gateway Security (Centralized)	Service Mesh Security (Decentralized)
Mechanism	The Gateway validates the JWT. It forwards raw User IDs to internal services via trusted HTTP headers (`X-User-Id`).	The Gateway forwards the JWT. Every single Microservice uses Spring Security to independently cryptographically verify the Token.
Trust Model	The internal services implicitly trust the Gateway. (Violation of strict Zero-Trust).	Strict Zero-Trust. Service B doesn't care who sent the request, it verifies the JWT itself.
mTLS	Optional / Handled by API Gateway.	Mandatory. Proxies (Envoy/Istio) encrypt traffic between `Service A <-> Service B`.
Implementation	Easy. Only the Gateway holds Spring Security logic. Downstream services just read the Headers.	Hard. Every service needs Spring Security `oauth2ResourceServer(jwt())` configured.
Performance	Fast. JWT parsed once at the Gateway.	Slower. JWT parsed at every hop. Each service fetches/caches JWKS keys.
Security Level	Medium. If Gateway is bypassed, internal services are exposed.	High. Even if an internal service is directly accessed, it rejects unauthenticated requests.

Real-World Production Scenario: Choosing an Architecture

```
Scenario: E-commerce platform with 12 microservices

Services: Auth Service, User Service, Product Service, Order Service,
          Payment Service, Notification Service, Search Service, etc.

Team discussion:

CTO: "We need to secure internal communication."

Option 1 -- API Gateway Pattern (chosen for v1.0):
  Pros:
    - Quick to implement; only Gateway has Spring Security
    - Lower latency (JWT parsed once)
  Cons:
    - If someone bypasses Gateway (e.g. direct Kubernetes port-forward),
      services are unprotected
```

```
  Decision: Acceptable for MVP, plan migration to Zero-Trust later

Option 2 -- Zero-Trust Pattern (target for v2.0):
  Pros:
    - Every service independently validates JWT
    - Even Kubernetes pod compromise doesn't spread
  Cons:
    - Every service needs oauth2ResourceServer configured
    - JWKS caching must be tuned for performance
  Decision: Migrate after product-market fit is established
```

12.2 How It Works (Strict Zero-Trust Pattern)

```
Step 1: Client logs in via External Auth Server (e.g., Keycloak)
        POST /realms/myapp/protocol/openid-connect/token
        Returns JWT: { accessToken, refreshToken }

Step 2: Client sends API request with JWT
        GET /api/orders
        Authorization: Bearer eyJhbGci...

Step 3: Spring Cloud Gateway intercepts
        - Extracts JWT from header
        - Validates signature against JWKS (Does NOT call Auth Server)
        - If valid: forwards request with original Authorization header
attached.

Step 4: Downstream Microservice (Order Service) receives request
        - Spring Security intercepts the incoming request.
        - Fetches JWKS Public Keys from cache.
        - Cryptographically verifies the JWT payload again!
        - Verifies the Token Scope (e.g. `hasAuthority('SCOPE_read:orders')`)
```

Step-by-Step: Configuring Zero-Trust in Each Microservice

Every microservice in a Zero-Trust architecture needs this configuration:

```yaml
# application.yml for Order Service
spring:
  security:
    oauth2:
      resourceserver:
```

```
        jwt:
          issuer-uri: https://keycloak.internal.company.com/realms/production
          jwk-set-uri:
https://keycloak.internal.company.com/realms/production/protocol/openid-
connect/certs
```

```
// SecurityConfig.java for Order Service
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class OrderServiceSecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
        return http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt ->
jwt.jwtAuthenticationConverter(keycloakRoleConverter()))
            )
            .build();
    }

    // Map Keycloak's realm_access.roles to Spring Security authorities
    @Bean
    public JwtAuthenticationConverter keycloakRoleConverter() {
        JwtAuthenticationConverter converter = new
JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(jwt -> {
            Map<String, Object> realmAccess =
jwt.getClaimAsMap("realm_access");
            if (realmAccess == null) return Collections.emptyList();
            List<String> roles = (List<String>) realmAccess.get("roles");
            return roles.stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
                .collect(Collectors.toList());
        });
        return converter;
```

```
    }
}
```

12.3 API Gateway JWT Filter (Centralized Trust Pattern)

If deciding to NOT use strict Zero-Trust for performance reasons, the Gateway parses the JWT and forwards the data as downstream trusted HTTP Headers.

```java
@Component
public class JwtGatewayFilter implements GatewayFilter {

    @Autowired private JwtService jwtService;

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
chain) {
        String authHeader = exchange.getRequest()
            .getHeaders().getFirst("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        String token = authHeader.substring(7);
        try {
            String userId = jwtService.extractSubject(token);
            List<String> roles = jwtService.extractRoles(token);

            // Forward user info to downstream Spring Boot apps via raw Headers
            ServerHttpRequest modifiedRequest = exchange.getRequest()
                .mutate()
                .header("X-Forwarded-User", userId)
                .header("X-Forwarded-Roles", String.join(",", roles))
                .build();

            return chain.filter(
                exchange.mutate().request(modifiedRequest).build());
        } catch (JwtException e) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
```

```
    }
}
```

Internal Working: How Downstream Services Read Gateway Headers

When using the Centralized Trust Pattern, downstream services don't need Spring Security at all. They simply read the trusted headers:

```
// In downstream Order Service (no Spring Security dependency needed!)
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    @GetMapping
    public List<Order> getMyOrders(
            @RequestHeader("X-Forwarded-User") String userId,
            @RequestHeader("X-Forwarded-Roles") String roles) {

        // Trust the Gateway's headers implicitly
        if (!roles.contains("ROLE_USER")) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN);
        }
        return orderService.findByUserId(Long.parseLong(userId));
    }
}
```

Security Pitfall: Header Spoofing Attack

```
CRITICAL VULNERABILITY in the Centralized Trust Pattern:

If an attacker bypasses the API Gateway (e.g., via Kubernetes port-forward
or a misconfigured network policy), they can directly call the Order Service
with FORGED headers:

  curl http://order-service:8080/api/orders \
    -H "X-Forwarded-User: admin-user-id" \
    -H "X-Forwarded-Roles: ROLE_ADMIN"

The Order Service blindly trusts these headers = FULL ADMIN ACCESS!

MITIGATION:
1. Kubernetes NetworkPolicies: Only allow traffic FROM the Gateway pod
2. mTLS: Verify client certificates on downstream services
```

```
3. Add a shared secret header that only the Gateway knows
4. Or: Switch to Zero-Trust (each service validates JWT independently)
```

12.4 Inter-Service Authentication (mTLS / Service Accounts)

If `Order Service` needs to talk to `Payment Service` *without* a user Context (e.g., a background batch job), how does `Order Service` authenticate?

Option 1: OAuth2 Client Credentials Flow The `Order Service` has its own `client_id` and `client_secret`. It calls the Auth Server to request an access token with machine-level scopes (e.g. `process:payments`). It attaches this token to its FeignClient calls.

Option 2: Mutual TLS (mTLS) Handled infra-level by Istio or AWS App Mesh. Both Microservices present an X.509 Certificate. The Transport Layer verifies the identity before a single byte of HTTP data is sent. Spring Boot is completely unaware of the security!

Deep Concept Explanation: OAuth2 Client Credentials Flow

```
Scenario: Order Service needs to call Payment Service to process a refund.
          There is no logged-in user -- this is a scheduled batch job at 2 AM.

Step 1: Order Service calls Keycloak with its OWN credentials
        POST /realms/production/protocol/openid-connect/token
        grant_type=client_credentials
        client_id=order-service
        client_secret=<order-service-secret>
        scope=payment:process payment:refund

Step 2: Keycloak returns a machine-level JWT
        {
          "sub": "order-service",
          "scope": "payment:process payment:refund",
          "iss": "https://keycloak.company.com/realms/production",
          "exp": 1710230400
        }

Step 3: Order Service calls Payment Service with this JWT
        POST http://payment-service/api/payments/refund
        Authorization: Bearer <machine-jwt>
        Body: { orderId: 42, amount: 99.99 }
```

```
Step 4: Payment Service validates the JWT using JWKS
        Checks: hasAuthority("SCOPE_payment:refund")? YES -> Process refund
```

Step-by-Step: Implementing Client Credentials in Spring Boot

```
// In Order Service: Configure WebClient with OAuth2 Client Credentials
@Configuration
public class OAuth2ClientConfig {

    @Bean
    public WebClient paymentServiceClient(
            ReactiveClientRegistrationRepository clientRegRepo,
            ServerOAuth2AuthorizedClientRepository authClientRepo) {

        ServerOAuth2AuthorizedClientExchangeFilterFunction oauth2Filter =
            new ServerOAuth2AuthorizedClientExchangeFilterFunction(
                clientRegRepo, authClientRepo);

        oauth2Filter.setDefaultClientRegistrationId("payment-service-client");

        return WebClient.builder()
            .baseUrl("http://payment-service:8080")
            .filter(oauth2Filter)  // Automatically attaches Bearer token!
            .build();
    }
}

// application.yml
spring:
  security:
    oauth2:
      client:
        registration:
          payment-service-client:
            provider: keycloak
            client-id: order-service
            client-secret: ${ORDER_SERVICE_CLIENT_SECRET}
            authorization-grant-type: client_credentials
            scope: payment:process,payment:refund
        provider:
          keycloak:
            token-uri:
https://keycloak.company.com/realms/production/protocol/openid-connect/token
```

Part 13: Security Best Practices & Defense

13.1 Cross-Origin Resource Sharing (CORS) Architecture

Why CORS exists: Browsers prevent malicious website `evil.com` from making an AJAX request to your bank `secure-bank.com` and reading the response.

Deep Concept Explanation

WHAT: CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts HTTP requests made from JavaScript running on Domain A to an API on Domain B. The browser enforces this by checking response headers from Domain B before allowing the JavaScript to read the response.

WHY it exists: Without CORS, any website could make AJAX requests to any other website. If you're logged into your bank at `bank.com` and visit `evil.com`, evil.com's JavaScript could call `bank.com/api/transfer` -- and the browser would attach your bank's cookies! CORS prevents `evil.com` from reading the response, but the request might still be sent (hence CSRF protection is also needed).

WHEN CORS becomes relevant:

- React SPA on `http://localhost:3000` calling Spring Boot on `http://localhost:8080` (different ports = different origins)
- Production frontend on `https://app.company.com` calling API on `https://api.company.com` (different subdomains = different origins)
- Mobile apps DO NOT enforce CORS (CORS is browser-only)

WHERE CORS is NOT relevant:

- Server-to-server calls (FeignClient, RestTemplate, WebClient) -- CORS is browser-only
- Mobile app API calls -- native HTTP clients don't enforce CORS
- Same-origin deployments -- when frontend and backend share the same domain and port

---

The Architect's View: The Preflight OPTIONS Request

When an SPA (React/Angular) running on `http://localhost:3000` tries to call `https://api.myapp.com`, the browser automatically stops the `POST` request.

Instead, the browser silently sends an `OPTIONS` HTTP request demanding the server answer: "*Are you willing to accept a POST request from localhost:3000?*"

If your Spring Security Filter Chain blocks the `OPTIONS` request with a 401 Unauthorized, CORS will fail! The Preflight request DOES NOT contain an Authorization header.

Internal Working: What the Browser Actually Does

```
1. JavaScript makes: fetch('https://api.myapp.com/orders', { method: 'POST' })
2. Browser detects: Cross-Origin request (different origin)
3. Browser checks: Is this a "Simple Request"?
   - Simple Request = GET/HEAD/POST with standard headers only
   - If simple: Browser sends the request directly with Origin header
   - If NOT simple (has Authorization header, PUT/DELETE, custom headers):

4. Browser sends PREFLIGHT request:
   OPTIONS /orders HTTP/1.1
   Host: api.myapp.com
   Origin: http://localhost:3000
   Access-Control-Request-Method: POST
   Access-Control-Request-Headers: Authorization, Content-Type

5. Server responds:
   200 OK
   Access-Control-Allow-Origin: http://localhost:3000
   Access-Control-Allow-Methods: GET, POST, PUT, DELETE
   Access-Control-Allow-Headers: Authorization, Content-Type
   Access-Control-Max-Age: 3600   <- Cache this preflight for 1 hour

6. Browser sees: Preflight passed! Now sends the ACTUAL request:
   POST /orders HTTP/1.1
   Host: api.myapp.com
   Origin: http://localhost:3000
   Authorization: Bearer eyJhbG...
   Content-Type: application/json
```

Proper Spring Setup: `CorsFilter` Before Security

```java
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    // NEVER use "*" in production if you allow credentials
    config.setAllowedOrigins(List.of("https://myapp.com",
```

```
"http://localhost:3000"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
    config.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-
Requested-With"));

    // Required if the frontend sends Cookies or Basic Auth credentials
    config.setAllowCredentials(true);

    // Cache the preflight response for 1 hour so the browser doesn't send
infinite OPTIONS requests
    config.setMaxAge(3600L);

    UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", config);
    return source;
}

// In SecurityFilterChain:
http.cors(cors -> cors.configurationSource(corsConfigurationSource()));
```

Security Pitfalls: CORS Mistakes

Mistake	Risk	Fix
`setAllowedOrigins(List.of("*"))` with `setAllowCredentials(true)`	Browser blocks the request (invalid config). You cannot use wildcard with credentials.	List specific origins explicitly
Not including `"OPTIONS"` in allowed methods	Preflight requests fail with 403	Always include OPTIONS
Setting CORS at `@CrossOrigin` on each Controller	Inconsistent, easy to forget on new endpoints	Centralize CORS in `SecurityConfig` using `CorsConfigurationSource`
Not setting `Access-Control-Max-Age`	Browser sends a preflight for EVERY single API call	Set `maxAge(3600)` -- cache for 1 hour

Monolithic vs Microservices: CORS Configuration

```
MONOLITHIC:
  Configure CORS once in the single SecurityConfig.
```

```
  Frontend URL is known and static.

MICROSERVICES:
  Option A: Configure CORS ONLY at the API Gateway level.
            Individual microservices don't need CORS because browsers
            only talk to the Gateway.

  Option B: Each microservice configures CORS independently.
            Necessary if services are exposed directly (no Gateway).

  Recommended: Option A -- centralize CORS at the Gateway.
```

13.2 Cross-Site Request Forgery (CSRF) Prevention

Cause: An attacker tricks a user's browser into executing a state-changing request (POST/DELETE) against a session-authenticated app where the user is already logged in. The browser blindly attaches the JSESSIONID cookie.

Architecture Type	CSRF Risk	Solution
Form-Login MVC	High Risk. Browsers auto-attach Cookies.	Must Enable CSRF. Spring inserts an anti-forgery hidden `_csrf` token in HTML forms.
REST API (Cookie Auth)	High Risk. SPAs using HttpOnly Session cookies.	Use `CookieCsrfTokenRepository` to inject the CSRF token into a Javascript-readable cookie.
REST API (JWT Auth)	Zero Risk. Browsers do NOT auto-attach Authorization Headers.	Safe to completely `.disable()` CSRF in Spring Security.

Deep Concept Explanation: How CSRF Works Internally

```
The Attack:

1. User logs into bank.com (gets JSESSIONID cookie for bank.com)
2. User visits evil.com (still has bank.com cookie in browser)
3. evil.com has hidden HTML:
   <form action="https://bank.com/transfer" method="POST">
     <input type="hidden" name="to" value="attacker-account">
     <input type="hidden" name="amount" value="10000">
   </form>
   <script>document.forms[0].submit();</script>
```

```
4. Browser submits POST to bank.com WITH the JSESSIONID cookie (auto-attached!)
5. Bank server sees valid session cookie -> processes the transfer!
6. $10,000 gone to attacker's account!

Spring Security's Defense:

1. Spring generates a random token: csrf_token = "abc123xyz"
2. Spring embeds it in the HTML form: <input type="hidden" name="_csrf"
value="abc123xyz">
3. When the form is submitted, Spring checks: does _csrf match the session's
token?
4. evil.com CANNOT read this token (Same-Origin Policy prevents it)
5. evil.com's forged form doesn't include _csrf -> Spring REJECTS the request
```

Real-World Production Scenario: CSRF in SPA + Cookie Architecture

```
Scenario: React SPA uses HttpOnly cookies for authentication (not JWT headers).
          The JSESSIONID cookie is auto-attached by the browser.
          CSRF protection is NEEDED!

Spring Boot Configuration:
  .csrf(csrf -> csrf
      .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
      // Sets a non-HttpOnly XSRF-TOKEN cookie that JavaScript CAN read
  )

React Frontend:
  // Read the XSRF-TOKEN cookie set by Spring
  const csrfToken = document.cookie
    .split('; ')
    .find(row => row.startsWith('XSRF-TOKEN='))
    ?.split('=')[1];

  // Include it in the X-XSRF-TOKEN header for POST/PUT/DELETE requests
  fetch('/api/orders', {
    method: 'POST',
    headers: { 'X-XSRF-TOKEN': csrfToken },
    body: JSON.stringify(order)
  });
```

13.3 Security Headers

By default, Spring Security automatically injects a robust set of security headers into every HTTP Response.

```
HTTP/1.1 200 OK
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000 ; includeSubDomains
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
```

What do they do?

1. **Strict-Transport-Security (HSTS):** Browsers caching that this domain MUST be contacted via HTTPS, thwarting downgrade/SSL-stripping attacks.
2. **X-Frame-Options: DENY:** Prevents "Clickjacking" (an attacker embedding your site in a hidden `<iframe>` to trick users into clicking buttons).
3. **X-Content-Type-Options: nosniff:** Prevents browsers from guessing the MIME type, thwarting script execution disguised as image files.

Deep Dive: Content Security Policy (CSP)

Spring Security does NOT add a Content-Security-Policy (CSP) header by default because it varies wildly per application. But in production, you SHOULD add one:

```java
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .headers(headers -> headers
            .contentSecurityPolicy(csp -> csp
                .policyDirectives(
                    "default-src 'self'; " +
                    "script-src 'self' https://cdn.jsdelivr.net; " +
                    "style-src 'self' https://fonts.googleapis.com; " +
                    "img-src 'self' data: https://images.myapp.com; " +
                    "connect-src 'self' https://api.myapp.com; " +
                    "font-src 'self' https://fonts.gstatic.com; " +
                    "frame-src 'none'"
                ))
        )
        .build();
}
```

This CSP header tells the browser:

- Only load scripts from `self` and `cdn.jsdelivr.net`
- Only load images from `self`, `data:` URIs, and `images.myapp.com`
- Disallow all `<iframe>` embedding (`frame-src 'none'`)
- If an attacker injects `<script src="evil.com/malware.js">`, the browser BLOCKS it!

Part 14: Production Security Challenges

14.1 Token Leaks & JWT Revocation Logic

The Problem: JWTs are stateless. If an access token with a 24-hour lifespan is stolen by a hacker, there is no "session" to delete on the server. The token is valid cryptographically!

The Architect's Mitigations:

1. **Short-Lived Access Tokens:** Keep them at 10-15 minutes max. Use Refresh Tokens to handle silent renewals.
2. **Redis Blacklisting:** Upon user logout, add the `jti` (JWT ID claim) to a distributed Redis cache with a Time-To-Live (TTL) matching the token's remaining time. Inside your `JwtFilter`, add one `redisTemplate.hasKey("blacklist:" + jti)` lookup.

```
// If true, the user clicked "Logout" before the token's natural expiration.
if (redisTemplate.hasKey("blacklist:" + claims.getId())) {
    throw new JwtException("Token has been revoked/blacklisted.");
}
```

Deep Concept Explanation: Token Revocation at Scale

The Challenge: In a microservices ecosystem handling 50,000 requests/second, checking Redis on EVERY request adds latency and load.

Architect's Approach: Layered Revocation Strategy

```
Layer 1: Access Token Expiration (10-15 minutes)
  - Most revocations are handled naturally by short TTLs
  - If a token is stolen, damage window is only 15 minutes
  - No Redis check needed for most requests!

Layer 2: Refresh Token Blacklisting (upon explicit logout)
  - When user logs out: blacklist the refresh token in Redis
  - The access token will expire naturally in < 15 minutes
  - Refresh attempts will fail -> user is effectively logged out

Layer 3: Emergency Token Revocation (security incident)
  - Mass revocation: rotate the JWT signing key
  - ALL existing tokens become invalid immediately
```

```
  - Users must re-authenticate (acceptable during a breach)

Layer 4: Bloom Filter for High-Throughput Blacklists
  - Instead of Redis lookup per request, use a Bloom Filter
  - Probabilistic data structure: "definitely NOT blacklisted" (fast)
    or "MIGHT be blacklisted" (check Redis to confirm)
  - Reduces Redis calls by ~99% for non-revoked tokens
```

Real-World Production Scenario: Handling a Token Leak Incident

```
Scenario: DevOps discovers that access tokens are being logged in plain text
          by the API Gateway's access logger (a misconfigured logging pipeline).
          Potentially thousands of tokens are exposed in log files.

Incident Response:
  1. IMMEDIATE: Rotate the JWT signing key on the Auth Server
     - All existing access tokens fail cryptographic validation
     - All users are forced to re-login (access disruption = 2 minutes)

  2. IMMEDIATE: Purge the exposed log files and fix the logging config

  3. POST-INCIDENT: Implement token fingerprinting
     - Bind each token to the client's IP + User-Agent hash
     - If someone replays the token from a different IP -> reject

  4. POST-INCIDENT: Set Access Token TTL to 5 minutes (from 15)
     - Shorter damage window for future incidents
```

14.2 Session Fixation Defense

Cause: An attacker visits your site, gets an anonymous `JSESSIONID=123`. The attacker sends a link to the victim: `http://myapp.com/login?JSESSIONID=123`. The victim clicks it and logs in. The server considers `123` authenticated. The attacker now refreshes their own tabs and assumes the victim's identity!

Spring's Defense: Spring Security natively runs `sessionFixation().migrateSession()`. The moment authentication succeeds, Spring generates an entirely new random `JSESSIONID` and invalidates the old one.

Internal Working: Session Fixation Strategies

```
.sessionManagement(session -> session
    // Option 1: migrateSession (DEFAULT & RECOMMENDED)
    // Creates a new session and copies all attributes from the old one
    .sessionFixation().migrateSession()

    // Option 2: newSession
    // Creates a brand new session, does NOT copy attributes
    // Use when old session might contain attacker-injected data
    .sessionFixation().newSession()

    // Option 3: changeSessionId (Servlet 3.1+)
    // Keeps the same session but changes the ID
    // Fastest option, but session object is reused
    .sessionFixation().changeSessionId()

    // Option 4: none (NEVER use in production!)
    // Disables session fixation protection entirely
    .sessionFixation().none()
)
```

14.3 Account Lockout and Dictionary Attacks

An attacker launches a script testing millions of common passwords against an administrator's email.

Mitigation Layering:

1. **Rate Limiting:** Throttle API requests via API Gateway (`Limit 10 POST /login per IP per min`).
2. **Account Lockout:** Track failed attempts in the database. `failed_attempts += 1` . After 5 failures, set `locked_until = now() + 30 mins` . Spring's `isAccountNonLocked()` checks this.
3. **CAPTCHA/MFA Integration:** If `failed_attempts > 3` , force the frontend to display a Google reCAPTCHA.

Step-by-Step: Complete Account Lockout Implementation

```
// Step 1: Add lockout fields to the User entity
@Entity
public class UserEntity {
    @Id @GeneratedValue
```

```java
    private Long id;
    private String email;
    private String passwordHash;

    private int failedAttempts = 0;
    private LocalDateTime lockedUntil;

    public boolean isAccountLocked() {
        return lockedUntil != null &&
LocalDateTime.now().isBefore(lockedUntil);
    }
}

// Step 2: Create a service to track login attempts
@Service
@RequiredArgsConstructor
public class LoginAttemptService {

    private final UserRepository userRepository;
    private static final int MAX_ATTEMPTS = 5;
    private static final int LOCK_DURATION_MINUTES = 30;

    @Transactional
    public void loginFailed(String email) {
        userRepository.findByEmail(email).ifPresent(user -> {
            int attempts = user.getFailedAttempts() + 1;
            user.setFailedAttempts(attempts);

            if (attempts >= MAX_ATTEMPTS) {

user.setLockedUntil(LocalDateTime.now().plusMinutes(LOCK_DURATION_MINUTES));
            }
            userRepository.save(user);
        });
    }

    @Transactional
    public void loginSucceeded(String email) {
        userRepository.findByEmail(email).ifPresent(user -> {
            user.setFailedAttempts(0);
            user.setLockedUntil(null);
            userRepository.save(user);
        });
    }
}

// Step 3: Hook into Spring Security's authentication events
@Component
```

```java
@RequiredArgsConstructor
public class AuthenticationEventListener {

    private final LoginAttemptService loginAttemptService;

    @EventListener
    public void
onAuthenticationFailure(AuthenticationFailureBadCredentialsEvent event) {
        String email = (String) event.getAuthentication().getPrincipal();
        loginAttemptService.loginFailed(email);
    }

    @EventListener
    public void onAuthenticationSuccess(AuthenticationSuccessEvent event) {
        String email = event.getAuthentication().getName();
        loginAttemptService.loginSucceeded(email);
    }
}

// Step 4: Check lock status in CustomUserDetailsService
@Override
public UserDetails loadUserByUsername(String email) {
    UserEntity user = userRepository.findByEmail(email)
        .orElseThrow(() -> new UsernameNotFoundException("Not found"));

    if (user.isAccountLocked()) {
        throw new LockedException("Account is locked until " +
user.getLockedUntil());
    }

    return new CustomUserDetails(user);
}
```

Monolithic vs Microservices: Account Lockout

Aspect	Monolithic	Microservices
Storage	Same DB as user table	Auth Service's dedicated DB (only Auth Service manages lockouts)
Rate Limiting	Implement in Spring Security filter	Implement at API Gateway level (e.g., Spring Cloud Gateway `RequestRateLimiter` with Redis)
Distributed tracking	Single server, simple in-memory counter	Use Redis-based counter shared across all instances of Auth Service

Notification	Email service is in same process	Publish `AccountLockedEvent` to Kafka -> Notification Service sends email

---

Part 15: Complete Spring Boot Security Project Skeleton

15.1 Project Structure

```
secure-enterprise-app/
+-- src/main/java/com/secureapp/
|   +-- SecureAppApplication.java
|   +-- config/
|   |   +-- CorsConfig.java               # Centralized CORS
|   |   +-- MethodSecurityConfig.java     # Enables @PreAuthorize
|   |   +-- OpenApiConfig.java            # Swagger annotations
|   +-- controller/
|   |   +-- AuthController.java           # Returns JWT payloads
|   +-- entity/
|   |   +-- UserEntity.java               # JPA Entity
|   +-- security/
|   |   +-- SecurityConfig.java           # FilterChain definition
|   |   +-- JwtService.java               # Token generation/verification via
Nimbus
|   |   +-- JwtAuthenticationFilter.java  # The interceptor proxy
|   |   +-- CustomUserDetails.java        # Wrapper for UserEntity
|   |   +-- CustomUserDetailsService.java # DB Lookup logic
|   +-- exception/
|       +-- SecurityExceptionHandler.java # Customizes 401/403 responses
+-- src/main/resources/
    +-- application.yml
```

Architect's View: Microservices Security Project Structure

```
enterprise-microservices/
+-- api-gateway/
|   +-- src/main/java/com/gateway/
|   |   +-- GatewayApplication.java
|   |   +-- config/
|   |   |   +-- GatewaySecurityConfig.java    # JWKS validation
|   |   |   +-- CorsConfig.java               # Centralized CORS
|   |   |   +-- RateLimitConfig.java          # Redis-based rate limiting
```

```
|   |   +-- filter/
|   |       +-- JwtGatewayFilter.java         # Extracts user info from JWT
|   +-- src/main/resources/
|       +-- application.yml                    # Route definitions
|
+-- auth-service/
|   +-- src/main/java/com/auth/
|   |   +-- AuthServiceApplication.java
|   |   +-- controller/
|   |   |   +-- AuthController.java            # /login, /register, /refresh
|   |   +-- security/
|   |   |   +-- SecurityConfig.java
|   |   |   +-- JwtService.java                # Signs tokens with PRIVATE key
|   |   |   +-- CustomUserDetailsService.java
|   |   +-- entity/
|   |       +-- UserEntity.java
|   +-- src/main/resources/
|       +-- application.yml
|       +-- keys/
|           +-- private-key.pem               # RS256 private key (GUARDED!)
|           +-- public-key.pem                # Published via JWKS endpoint
|
+-- order-service/
|   +-- src/main/java/com/orders/
|   |   +-- OrderServiceApplication.java
|   |   +-- config/
|   |   |   +-- SecurityConfig.java           # oauth2ResourceServer(jwt())
|   |   +-- controller/
|   |       +-- OrderController.java
|   +-- src/main/resources/
|       +-- application.yml                   # jwk-set-uri pointing to auth-
service
|
+-- docker-compose.yml                        # Orchestrates all services
```

15.2 Spring Boot Starter Dependencies

```
<dependencies>
    <!-- Core Spring Security -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
```

```
    <!-- OAuth2 Resource Server for JWT Parsing -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
    </dependency>
    <!-- Web for REST APIs -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

Full Dependencies for Production-Grade Security Setup

```
<dependencies>
    <!-- Core -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- JWT (Choose ONE: jjwt OR spring-oauth2-resource-server) -->
    <!-- Option A: Manual JWT with jjwt -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.5</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.5</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.12.5</version>
        <scope>runtime</scope>
    </dependency>
```

```
    <!-- Option B: OAuth2 Resource Server (auto JWKS validation) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
    </dependency>

    <!-- Database -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- Redis for Token Blacklisting & Rate Limiting -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-redis</artifactId>
    </dependency>

    <!-- Validation -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
</dependencies>
```

Part 16: Architect-Level Security Interview Questions (100+)

1. Fundamentals & Spring Internals (Questions 1-20)

Q1. How does a Servlet Filter (which Tomcat runs) know about a Spring Bean like your custom JwtFilter? *Architect's Answer:* Tomcat doesn't know about Spring. Spring bridges this gap using `DelegatingFilterProxy`, which is registered as a standard Servlet Filter but delegates all work to the `FilterChainProxy` Spring Bean, which subsequently houses our internal Security Filter Chains.

Deep Explanation: When Spring Boot starts, `SecurityFilterAutoConfiguration` registers `DelegatingFilterProxy` with the embedded Servlet Container. This proxy looks up a bean named `springSecurityFilterChain` from the Spring `ApplicationContext`. That bean is `FilterChainProxy`, which maintains a `List<SecurityFilterChain>`. For each request, `FilterChainProxy` iterates through the list and finds the first chain whose `securityMatcher()` matches the request URL.

Real-World Impact: This means if an exception is thrown inside the filter chain (before `DispatcherServlet`), your `@ControllerAdvice` will NOT catch it. You must handle security exceptions within the filter chain using `AuthenticationEntryPoint` and `AccessDeniedHandler`.

Q2. In a highly concurrent REST API, how does Spring ensure one thread doesn't read the SecurityContext of another thread? *Architect's Answer:* The `SecurityContextHolder` utilizes the `ThreadLocal` strategy. This binds the context securely to the executing thread.

Deep Explanation: Tomcat allocates a thread from its thread pool for each incoming HTTP request. `SecurityContextHolder` uses `ThreadLocal<SecurityContext>`, which gives each thread its own isolated copy. Even if 1,000 requests arrive simultaneously, Thread-1's `SecurityContext` (User A) is completely invisible to Thread-2's `SecurityContext` (User B).

Production Scenario: In a banking app handling 5,000 concurrent transactions, ThreadLocal guarantees that User A's account balance query never accidentally uses User B's authentication credentials.

Q3. If I spawn a heavy background task using `@Async`, why do I suddenly lose my Authentication object? How do you fix it? *Architect's Answer:* Because `@Async` spawns a new thread, and the `SecurityContext` is bound strictly to `ThreadLocal`. You fix it by wrapping your `Executor` or `ThreadPoolTaskExecutor` in a `DelegatingSecurityContextExecutorService`, which automatically copies the Context into the new thread before execution.

Real-World Scenario: An e-commerce platform sends order confirmation emails asynchronously. The email template includes "Dear {username}". Without context propagation, `SecurityContextHolder.getContext().getAuthentication()` returns `null` in the async thread, causing a `NullPointerException`.

Monolithic Fix:

```
@Bean
public Executor asyncExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.initialize();
    return new DelegatingSecurityContextExecutor(executor);
}
```

Microservices Fix: When publishing events to Kafka, serialize the JWT string into the Kafka message header. The consumer service extracts and validates the JWT to restore the SecurityContext.

Q4. What is the difference between `AuthenticationManager` and `AuthenticationProvider`? *Architect's Answer:* `AuthenticationManager` uses the Strategy Pattern. Its default implementation (`ProviderManager`) loops through a list of registered `AuthenticationProviders`. This allows a single application to support DB Login (`DaoAuthProvider`), API Key Login (`ApiKeyAuthProvider`), and OAuth2 simultaneously.

Deep Explanation: The `AuthenticationManager` interface has one method: `authenticate(Authentication)`. The `ProviderManager` (its main implementation) iterates through its list of `AuthenticationProvider`s, calling `provider.supports(tokenType)` on each one. Only providers that support the given token type attempt authentication. If none succeed, a `ProviderNotFoundException` is thrown.

Q5. Why should you never use `MD5` or `SHA-256` for password hashing? Why is `BCrypt` the standard? *Architect's Answer:* Standard hashing algorithms are fast by design, making them vulnerable to

Rainbow Table dictionaries and GPU Brute Forcing. BCrypt embeds a random 22-character salt (thwarting rainbow tables) and intentionally slows down the hashing process (e.g., via 4096 iterations) to mathematically prevent brute-force attacks from scaling.

Numbers that matter:

- MD5: 10 billion hashes/second on a modern GPU -> cracked in seconds
- SHA-256: 8 billion hashes/second -> cracked in seconds
- BCrypt (strength 12): 8 hashes/second -> cracking 1 password takes years

---

Q6. How do you migrate a legacy system consisting of 1 Million `MD5` user passwords to `BCrypt` with zero downtime? *Architect's Answer:* Use `DelegatingPasswordEncoder`. Set `BCrypt` as the default for new signups. Register the `MD5` encoder as a fallback. When a user logs in successfully using their legacy `MD5` password in memory, rewrite the password to DB immediately using `BCrypt`.

Step-by-Step Migration:

1. Database stores: `{MD5}5f4dcc3b5aa765d...` (old) and `{bcrypt}$2a$12$...` (new)
2. User logs in with password "secret123"
3. Spring sees `{MD5}` prefix -> uses MD5 encoder to verify
4. Verification succeeds -> Spring calls `UserDetailsPasswordService.updatePassword()`
5. Password is re-hashed: `{bcrypt}$2a$12$...` and saved to database
6. Next login: Spring uses BCrypt directly (MD5 never touched again)

Q7. Explain the exact difference between Authentication (AuthN) and Authorization (AuthZ). *Architect's Answer:* Authentication asks: "Who are you?" (proving identity via passwords/MFA). Authorization asks: "What are you allowed to do?" (checking Roles, Scopes, and Authorities). AuthN always precedes AuthZ.

Production Example: In a hospital system:

- AuthN: Dr. Smith presents her badge + PIN -> system confirms she IS Dr. Smith
- AuthZ: Dr. Smith tries to access Patient X's records -> system checks if Dr. Smith is assigned to Patient X -> if NOT, access denied (403), despite valid authentication

---

Q8. What happens internally when you call `http.cors().configurationSource(..)`? *Architect's Answer:* Spring injects a `CorsFilter` *before* the security checks. When a browser sends an HTTP `OPTIONS` preflight request (which inherently lacks an Authentication header), the

`CorsFilter` intercepts it. If configured correctly, it responds with a 200 OK allowing the CORS flow, preventing the `AnonymousAuthenticationFilter` from rejecting the preflight request with a 401.

Common Production Bug: Developer configures CORS on the `@RestController` using `@CrossOrigin` but forgets that Spring Security's filter chain runs BEFORE the controller. The preflight OPTIONS request hits the security chain first and gets a 401 because it has no Authorization header.

Fix: Always configure CORS at the `SecurityFilterChain` level using `http.cors(cors -> cors.configurationSource(...))`, not at the controller level.

Q9. If you build a completely Stateless REST API utilizing JWTs, do you need CSRF protection? *Architect's Answer:* No. CSRF (Cross-Site Request Forgery) attacks rely on browsers automatically attaching `JSESSIONID` Cookies to forged requests. If you use stateless JWTs sent explicitly via the HTTP `Authorization: Bearer` header, the browser won't auto-send the token. You can safely `.disable()` CSRF.

But be careful: If you store the JWT in a Cookie (instead of the Authorization header), CSRF protection is STILL needed because browsers auto-attach cookies! The decision depends on WHERE you store the token:

- In-memory JavaScript variable / Authorization header -> CSRF disabled ✓
- HttpOnly Cookie -> CSRF enabled ✓ (use `CookieCsrfTokenRepository`)

Q10. How does the `ExceptionTranslationFilter` handle security errors? *Architect's Answer:* It acts as a safety net catching exceptions thrown deeper in the chain. If an `AuthenticationException` is caught, it triggers the `AuthenticationEntryPoint` (translating to a 401 HTTP response). If an `AccessDeniedException` is caught, it triggers the `AccessDeniedHandler` (translating to a 403 HTTP response).

Production Impact: If you don't customize these handlers for a REST API, Spring defaults to returning an HTML login page for 401 errors. Your SPA/mobile app receives HTML instead of JSON -> crashes.

---

2. JWT Architecture & OIDC (Questions 21-40)

Q11. Explain the architectural flaw of using Symmetric Signatures (HS256) for a JWT in a microservices ecosystem. *Architect's Answer:* Symmetric keys (`HMAC-SHA256`) use the exact same secret key to sign the token (Auth Server) and verify the token (Resource Server). If an attacker breaches

any single downstream microservice and reads the secret key from `application.yml`, they can forge valid "ADMIN" JWTs and compromise the entire cluster.

Monolithic vs Microservices:

- Monolithic: HS256 is acceptable because only ONE application holds the secret
- Microservices: HS256 is dangerous because the secret must be shared across all services

---

Q12. What is the solution? Explain JWKS and RS256. *Architect's Answer:* Use Asymmetric cryptography (`RS256`). The Auth Server holds the strictly guarded **Private Key** to sign the JWTs. The Auth server publishes the **Public Keys** via a JSON Web Key Set (JWKS) endpoint. Downstream Spring Boot APIs fetch the JWKS. If the downstream microservice is breached, the attacker only gets the Public Key, which cannot forge tokens.

Spring Boot Config (Zero secrets in downstream services):

```yaml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: https://auth-service/.well-known/jwks.json
```

Q13. How do you implement Zero-Downtime Key Rotation using JWKS? *Architect's Answer:* Generate a new Private/Public key pair. Add the new Public Key to the JWKS endpoint alongside the old one. The Auth Server starts signing new tokens with the new Private Key, explicitly setting the `kid` (Key ID) in the JWT Header. Spring Boot APIs see the new `kid`, fetch the updated JWKS, and continue validating. Once all old tokens expire, remove the old key from the JWKS list.

Why `kid` (Key ID) matters: The JWT header contains `"kid": "key-2024-03"`. When a microservice receives this JWT, it checks its cached JWKS for a key with this `kid`. If not found, it fetches the JWKS endpoint again. This is how Spring auto-discovers new keys without any deployment!

Q14. In OAuth2, what is the architectural distinction between a Role and a Scope? *Architect's Answer:* A **Role** (e.g. `ROLE_ADMIN`) is inherent to the User (RBAC), answering "What is this user allowed to do inside the API?". A **Scope** (e.g. `read:orders`) dictates what the *Client Application* is authorized to do *on behalf* of the user. Spring Security enforces scopes via `@PreAuthorize("hasAuthority('SCOPE_read:orders')")`.

Production Example: A user is an ADMIN in the CRM. Zapier integration has only `read:contacts` scope. Even though the user IS an admin, Zapier cannot delete contacts because the OAuth2 scope doesn't include `write:contacts`.

Q15. Is OAuth2 an Authentication Protocol? Explain. *Architect's Answer:* No, OAuth2 is purely a delegated **Authorization** framework. It allows a third-party app to access resources without sharing credentials. For actual **Authentication** (identity proving), we use **OpenID Connect (OIDC)**, an identity layer built on top of OAuth2 that provides an `ID Token` (JWT) containing user profiling data.

Key Difference:

- OAuth2 Access Token: `{ scope: "read:calendar" }` -> "This app can read your calendar" (Authorization)
- OIDC ID Token: `{ sub: "user123", name: "John", email: "j@mail.com" }` -> "This IS John" (Authentication)

Q16. How do you revoke a stateless JWT before its expiration? *Architect's Answer:* A JWT is cryptographically valid until its `exp` claim hits. The only way to securely revoke it early is to push its `jti` (JWT ID) to a highly available, high-speed distributed cache like Redis (with a TTL matching the token's remaining lifespan). The `JwtFilter` must assert `!redisTemplate.hasKey(jti)` on every request.

Alternative for emergency mass-revocation: Rotate the JWT signing key. All existing tokens become cryptographically invalid instantly. Every user must re-authenticate.

Q17. Why do we pair short-lived Access Tokens with long-lived Refresh Tokens? *Architect's Answer:* Because checking Redis on every request to see if an Access Token is revoked kills the performance of a stateless architecture. By making Access Tokens expire in 10 minutes, we limit the damage window if a token is stolen. The frontend uses the secure Refresh Token to silently request new Access Tokens. We only hit Redis to revoke Refresh Tokens upon explicit "Logout" requests.

Architect's Math:

- 50,000 requests/second x 1 Redis check per request = 50,000 Redis ops/second (expensive!)
- With short-lived access tokens: Redis only checked on refresh (every 10 minutes) = ~80 checks/second (negligible)

Q18. What is Token Replay, and how do you prevent it? *Architect's Answer:* An attacker intercepts a valid JWT and resends the exact same HTTP request to execute the action twice (e.g. transferring money). Mitigate this by enforcing **Idempotency Keys** on `POST` requests, preventing the server from processing the same transaction UUID twice.

Implementation:

```
@PostMapping("/transfer")
public ResponseEntity<TransferResult> transfer(
    @RequestHeader("X-Idempotency-Key") String idempotencyKey,
    @RequestBody TransferRequest request) {

    // Check if this exact transaction was already processed
    if (redisTemplate.hasKey("idempotency:" + idempotencyKey)) {
        return ResponseEntity.ok(cachedResult);
    }
    // Process transaction and cache the result
}
```

Q19. What is a "Confidential Client" vs a "Public Client" in OAuth2? *Architect's Answer:* A Confidential Client (e.g., a Spring Boot Backend) can securely store a `client_secret` out of reach of the user. A Public Client (e.g., a React SPA or iOS app) cannot securely hold secrets, as the code is delivered to the user's device. Public clients must use the **Authorization Code Flow with PKCE** to prove identity without a static secret.

Why PKCE (Proof Key for Code Exchange): Without PKCE, if an attacker intercepts the Authorization Code during the redirect, they can exchange it for tokens. PKCE adds a dynamically generated `code_verifier` (random string) -> hashed into `code_challenge`. The Auth Server verifies the code_verifier matches the code_challenge. The attacker cannot forge this because they never saw the original code_verifier.

Q20. How does the Client Credentials flow differ from Authorization Code flow? *Architect's Answer:* The Client Credentials flow is for Machine-to-Machine (Service-to-Service) communication where there is no human user involved. A backend service authenticates itself using its own `client_id` and `client_secret` to obtain a token.

Example: Order Service calls Payment Service at 2 AM for a scheduled batch refund. No user is logged in. Order Service uses its own `client_id` / `client_secret` to get a machine-level token with `payment:refund` scope.

---

3. Production & Zero-Trust Architecture (Questions 41-60)

Q21. Explain Perimeter Security vs Zero-Trust Architecture. *Architect's Answer:* Perimeter Security trusts any traffic that originates from inside the internal network (VPC/VPN). Zero Trust assumes the

network is already breached. Every single request, even those between internal Microservices running on the same Kubernetes Node, must be cryptographically authenticated and authorized.

Real-World Breach: In 2020, the SolarWinds attack demonstrated why Perimeter Security fails. Attackers compromised an internal network monitoring tool. Because internal traffic was trusted, the attackers moved laterally across thousands of systems undetected. With Zero-Trust, each service-to-service call would have required separate authentication.

Q22. How do you implement internal security using an API Gateway Pattern? *Architect's Answer:* The Gateway (e.g. Spring Cloud Gateway) parses and validates the JWT against the Keycloak JWKS server. If valid, the Gateway strips the JWT and attaches internal HTTP Headers (`X-User-Id` , `X-Roles`) before routing downstream. The downstream microservices *implicitly trust* the Gateway. Not strict Zero-Trust, but highly performant.

Security Warning: If an attacker can bypass the Gateway (e.g., via Kubernetes port-forward, misconfigured ingress, or a compromised internal service), they can spoof the `X-User-Id` header and impersonate any user!

Q23. How do you implement internal security using a Service Mesh/Zero-Trust Pattern? *Architect's Answer:* The Gateway blindly forwards the raw HTTP `Authorization: Bearer <jwt>` Header downstream. Every single internal Microservice is configured as a Spring Security `OAuth2ResourceServer` . Every microservice independently fetches the JWKS and cryptographically verifies the signature before processing the request. This represents strict Zero-Trust.

Q24. What is Mutual TLS (mTLS), and why is it required in true Zero-Trust? *Architect's Answer:* Standard TLS encrypts traffic, but only the Client verifies the Server's certificate. In Mutual TLS, BOTH the Client and the Server present certificates. When Microservice A calls Microservice B, they establish a cryptographically secure, encrypted tunnel proving each other's identities at the transport layer, effectively preventing Network Sniffing inside the VPC. Usually handled by Istio/Envoy sidecars.

How Istio implements mTLS:

- Istio injects an Envoy sidecar proxy into every Kubernetes pod
- When Pod A calls Pod B, their Envoy sidecars handle the mTLS handshake
- Spring Boot is completely unaware -- it thinks it's making plain HTTP calls
- The application code doesn't change AT ALL

Q25. How do you implement Session Fixation defense in a monolithic, session-based app? *Architect's Answer:* When a user authenticates successfully, immediately invalidate their pre-login `JSESSIONID` and generate a completely new, cryptographically random `JSESSIONID` . Spring Security handles this natively via `sessionFixation().migrateSession()` .

Q26. How do you prevent a user from logging into the same account simultaneously from 10 different laptops? *Architect's Answer:* Register a `SessionRegistry` bean and configure `.sessionManagement().maximumSessions(1)`. Note: If deployed across multiple EC2 instances behind a Load Balancer, local memory execution fails. You must implement Spring Session backed by a centralized Redis cluster (`@EnableRedisHttpSession`).

Q27. Design a Spring Security RBAC system to authorize access based on dynamic database attributes (ABAC). *Architect's Answer:* Utilize Aspect-Oriented Method Security via `@PreAuthorize`. Instead of a static role check, use SpEL to enforce Attribute-Based Access Control logic executing before the DAO method: `@PreAuthorize("hasRole('ADMIN') or #document.ownerId == authentication.principal.id")`.

Advanced Scenario: In a document management system, a manager can only delete documents created by employees in their own department:

```
@PreAuthorize("hasRole('MANAGER') and
@departmentService.isInSameDept(authentication.principal.id, #docId)")
public void deleteDocument(Long docId) { ... }
```

Q28. What is an OAuth2 "Token Exchange" in a microservices ecosystem? *Architect's Answer:* When Service A receives a user JWT and needs to call Service B. Instead of passing the user's raw token, Service A calls the Auth Server to exchange the token for a new token scoped *specifically* for Service B, limiting access privileges (Defense in Depth).

Why not just forward the original token? If Order Service receives a token with scopes `[order:read, order:write, payment:read, user:admin]` and forwards it to Payment Service, Payment Service sees ALL scopes, even ones irrelevant to payments. Token Exchange narrows the scope to only `[payment:read]`.

Q29. What happens if your Spring Boot app's `application.yml` contains `spring.security.oauth2.resourceserver.jwt.jwk-set-uri`, but the Keycloak server crashes? *Architect's Answer:* Spring Security caches the JWKS public keys. If Keycloak crashes, Spring Security can continue to securely validate incoming JWTs independently until the cache TTL expires or a token requires a new `kid` signature.

Production Resilience Tip: Configure JWKS cache timeout to a reasonable value (e.g., 1 hour). This ensures your API remains operational even if the Auth Server has brief downtime. But also monitor Keycloak health checks aggressively to catch outages early.

Q30. Explain the concept of "Defense in Depth" as applied to Spring Security. *Architect's Answer:* Do not rely entirely on one layer of security. Use HSTS Headers for transport safety. Use the API Gateway for rate limiting and coarse Token validation. Use Spring Security Method Annotations (`@PreAuthorize`) for fine-grained row-level business logic. Use mTLS for VPC tunneling. If the API Gateway is misconfigured, mTLS and internal RBAC annotations will catch the intruder.

The 5 Layers:

```
Layer 1: Network (AWS Security Groups, VPC, WAF)
Layer 2: Transport (HTTPS/TLS, mTLS for internal communication)
Layer 3: API Gateway (Rate limiting, IP blocking, JWT validation)
Layer 4: Application (Spring Security FilterChain, CORS, CSRF)
Layer 5: Business Logic (@PreAuthorize, data-level authorization, audit logging)
```

---

4. Advanced System Design Security Questions (Questions 61-80)

Q31. Design an authentication system that can handle 1 million concurrent users. *Architect's Answer:*

```
Architecture:
  - Auth Service: 5 instances behind NLB, connected to PostgreSQL (primary +
read replicas)
  - JWT with RS256: Auth Service signs tokens with private key
  - Access Token TTL: 10 minutes (stateless, no server-side storage)
  - Refresh Token: Stored in Redis cluster (3 nodes, high availability)
  - All other microservices: Validate JWT with cached JWKS public key
  - Zero Redis calls for normal API requests (only during token refresh)

Capacity Math:
  - 1M concurrent users x 1 request/sec = 1M requests/second
  - Each request: JWT validation via cached public key = ~1ms CPU (no I/O)
  - No database call per request (stateless JWT)
  - Redis hit only during refresh: 1M users / 600 seconds = ~1,600
refreshes/second
```

Q32. How would you design a multi-tenant security system where tenants can customize their own RBAC? *Architect's Answer:*

```
Design:
  - Each tenant has its own set of Roles and Permissions in the database
  - TenantAwareUserDetails carries tenantId
```

```
  - A @PreAuthorize custom evaluator loads tenant-specific permissions
  - The SecurityContext is enriched with tenant-level authorities

Implementation:
  @PreAuthorize("@tenantAuthz.hasPermission(authentication, 'order:delete')")

  @Component
  public class TenantAuthz {
    public boolean hasPermission(Authentication auth, String permission) {
      Long tenantId = ((TenantUserDetails) auth.getPrincipal()).getTenantId();
      return permissionRepo.existsByTenantIdAndUserIdAndPermission(
        tenantId, auth.getName(), permission);
    }
  }
```

Q33. How do you secure internal APIs so that only specific microservices can call them? *Architect's Answer:* Use OAuth2 Client Credentials with service-specific scopes. Order Service gets a token with scope `payment:process`. User Service gets a token with scope `user:read`. Payment Service validates: `@PreAuthorize("hasAuthority('SCOPE_payment:process')")`. Even if the User Service is compromised, it cannot call payment APIs because its token doesn't have the right scope.

Q34. How would you implement security audit logging for compliance (SOC2, HIPAA)? *Architect's Answer:*

```java
@Component
public class SecurityAuditListener {

    @Autowired private AuditLogRepository auditRepo;

    @EventListener
    public void onSuccess(AuthenticationSuccessEvent event) {
        auditRepo.save(AuditLog.builder()
            .event("LOGIN_SUCCESS")
            .userId(event.getAuthentication().getName())
            .ip(extractIp(event))
            .timestamp(Instant.now())
            .build());
    }

    @EventListener
    public void onFailure(AbstractAuthenticationFailureEvent event) {
        auditRepo.save(AuditLog.builder()
            .event("LOGIN_FAILURE")
            .userId((String) event.getAuthentication().getPrincipal())
```

```
            .reason(event.getException().getMessage())
            .ip(extractIp(event))
            .timestamp(Instant.now())
            .build());
    }
}
```

Q35. How do you handle token expiration gracefully in a Single Page Application? *Architect's Answer:* Implement a Silent Refresh mechanism. The Axios/Fetch interceptor catches 401 responses. Before showing an error, it calls the `/refresh` endpoint with the refresh token cookie. If successful, the original request is retried with the new access token. If the refresh also fails (token expired), redirect to the login page. The user never sees the intermediate 401.

---

5. Scenario-Based Interview Questions (Questions 81-100+)

Q36. Your production API is returning 403 for an admin user. How do you debug this? *Architect's Debugging Process:*

```
1. Enable debug logging: logging.level.org.springframework.security=DEBUG
2. Check logs for: "Checking authorization for GET /api/admin/users"
3. Look for: "Granted Authorities: [ROLE_admin]" -- note the casing!
4. The config uses: hasRole("ADMIN") which looks for "ROLE_ADMIN"
5. Database stores: "admin" (lowercase), authority becomes "ROLE_admin"
6. Fix: Either store "ADMIN" in DB or use hasRole("admin")
7. ALTERNATIVE FIX: Use case-insensitive comparison in custom voter
```

Q37. Your React SPA can't call the Spring Boot API. The browser console shows "CORS preflight failed." *Architect's Debugging Process:*

```
1. Check: Is the CORS configured at SecurityFilterChain level?
   (.cors(cors -> cors.configurationSource(...)) -- NOT @CrossOrigin on
controller)
2. Check: Does the CorsConfiguration include OPTIONS in allowedMethods?
3. Check: Is the Authorization header listed in allowedHeaders?
4. Check: Does setAllowedOrigins include the React URL (with port!)?
5. Check: If using credentials, you CANNOT use "*" as allowed origin
6. Check: Is the preflight cached? (setMaxAge to avoid repeated OPTIONS calls)
```

Q38. A pentester reports that your API returns different error messages for "user not found" vs "incorrect password." Why is this dangerous? *Architect's Answer:* This is a User Enumeration vulnerability. An attacker can determine which emails are registered by testing login attempts. Return the SAME generic message for both cases: "Invalid credentials." Spring Security's `DaoAuthenticationProvider` actually helps here -- it runs `PasswordEncoder.matches()` even when the user is not found (to equalize response time).

Q39. Your JWT-protected API works perfectly locally but fails in production behind an AWS ALB. Why? *Architect's Debugging Process:*

```
1. Check: Does the ALB strip the Authorization header? (Some ALB configs do
this)
2. Check: Is the ALB performing TLS termination? If so, the backend receives
HTTP.
   Spring's HSTS header says "use HTTPS" but the backend sees HTTP -> confusion
3. Fix: Trust the X-Forwarded-Proto header:
   server.forward-headers-strategy=NATIVE
4. Check: Is the ALB adding a health check that hits a secured endpoint? -> 401
   Fix: Expose /actuator/health as permitAll()
```

Q40. Design a security system that supports both API Key authentication for external integrations AND JWT authentication for mobile apps in the same Spring Boot application. *Architect's Answer:*

```java
// Two separate SecurityFilterChain beans!

@Bean @Order(1)
public SecurityFilterChain apiKeyChain(HttpSecurity http) throws Exception {
    return http
        .securityMatcher("/api/external/**")
        .addFilterBefore(apiKeyFilter,
UsernamePasswordAuthenticationFilter.class)
        .authorizeHttpRequests(auth -> auth.anyRequest().hasRole("API_CLIENT"))
        .build();
}

@Bean @Order(2)
public SecurityFilterChain jwtChain(HttpSecurity http) throws Exception {
    return http
        .securityMatcher("/api/**")
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated())
```

```
        .build();
}
// Order matters! /api/external/** is matched first by Chain 1.
// All other /api/** URLs are matched by Chain 2.
```

Part 17: Hands-on Practice Exercises

Exercise 1: Build JWT Authentication System (Symmetric)

Goal

Build a complete login system with Spring Boot + JWT from scratch.

Step-by-Step

1. Generate User entity and Repository.
2. Implement CustomUserDetailsService.
3. Build `JwtService` utilizing `jjwt-api` with a hardcoded Secret Key (`HS256`).
4. Build `JwtAuthenticationFilter` that extends `OncePerRequestFilter` .
5. Return tokens inside `AuthController` .

Detailed Implementation Checklist

```
[ ] Create Spring Boot project with: spring-boot-starter-security, web, jpa,
postgresql
[ ] Create UserEntity with: id, email, passwordHash, role, active, locked
[ ] Create UserRepository with: findByEmail(String email)
[ ] Create CustomUserDetails (wrapper around UserEntity)
[ ] Create CustomUserDetailsService (implements UserDetailsService)
[ ] Create JwtService with:
    [ ] generateAccessToken(UserDetails user) -> returns JWT string
    [ ] extractUsername(String token) -> returns email from sub claim
    [ ] isTokenValid(String token, UserDetails user) -> boolean
    [ ] getSigningKey() -> SecretKey from Base64 secret
[ ] Create JwtAuthenticationFilter (extends OncePerRequestFilter)
    [ ] Extract Bearer token from Authorization header
    [ ] Validate token and set SecurityContext
    [ ] Add shouldNotFilter() for public endpoints
[ ] Create SecurityConfig:
    [ ] SessionCreationPolicy.STATELESS
    [ ] CSRF disabled
    [ ] addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
    [ ] Custom AuthenticationEntryPoint returning JSON 401
[ ] Create AuthController:
    [ ] POST /api/auth/register -> hash password, save user
    [ ] POST /api/auth/login -> authenticate, return JWT
[ ] Create a protected endpoint: GET /api/users/me -> returns current user info
```

```
[ ] Test with Postman/cURL

Expected Test Flow:
  POST /api/auth/register -> 201 Created
  POST /api/auth/login -> 200 OK { "accessToken": "eyJ..." }
  GET /api/users/me (no token) -> 401 Unauthorized
  GET /api/users/me (with Bearer token) -> 200 OK { "email": "..." }
```

Exercise 2: OAuth2 Resource Server (Asymmetric / JWKS)

Goal

Set up a real Keycloak server and configure Spring Boot to validate JWTs via JWKS.

Step-by-Step

1. Run a local instance of Keycloak via Docker.
2. Create a Realm and a Test User.
3. Configure `application.yml` to point `jwk-set-uri` at the Keycloak Endpoint.
4. Protect a REST API using `@PreAuthorize("hasAuthority('SCOPE_email')")`.
5. Obtain a Token via Postman (Implicit Flow) and call the Spring Boot API.

Detailed Implementation Checklist

```
[ ] Run Keycloak: docker run -p 8180:8080 -e KEYCLOAK_ADMIN=admin -e
KEYCLOAK_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak start-dev
[ ] Login to Keycloak admin console: http://localhost:8180
[ ] Create a realm: "my-app-realm"
[ ] Create a client: "spring-boot-api" (confidential, Authorization Code)
[ ] Create a test user: "testuser" with password
[ ] Assign roles to the user: "user", "admin"
[ ] Note the JWKS endpoint: http://localhost:8180/realms/my-app-
realm/protocol/openid-connect/certs
[ ] Configure Spring Boot application.yml with jwk-set-uri
[ ] Create SecurityConfig with oauth2ResourceServer(jwt())
[ ] Create JwtAuthenticationConverter to map Keycloak roles
[ ] Create protected endpoint: GET /api/profile
[ ] Use Postman to get a token from Keycloak's token endpoint
[ ] Call GET /api/profile with the Bearer token
[ ] Verify role-based access: user without "admin" role gets 403 on admin
endpoint
```

Exercise 3: Role-Based Authorization & Session Management

Goal

Build a monolithic MVC application with session-based security, RBAC, and CSRF.

Step-by-Step

1. Build an MVC Form-Login application.
2. Configure `.maximumSessions(1)` to block concurrent logins.
3. Implement `CookieCsrfTokenRepository` to inject anti-forgery tokens.
4. Map `ROLE_ADMIN` and `ROLE_USER` to different URLs.
5. Create a Service method protected by `@PreAuthorize("hasRole('ADMIN')")`.

Detailed Implementation Checklist

```
[ ] Create Spring Boot project with: spring-boot-starter-security, web,
thymeleaf, jpa, h2
[ ] Create UserEntity with roles (ManyToMany relationship)
[ ] Create a custom login page (Thymeleaf template)
[ ] Configure SecurityFilterChain:
    [ ] Form login with custom login page
    [ ] Session management: maximumSessions(1), migrateSession()
    [ ] URL-based security: /admin/** requires ROLE_ADMIN
    [ ] CSRF enabled (default for form login)
[ ] Create AdminController: GET /admin/dashboard -> only ADMIN can access
[ ] Create UserController: GET /user/profile -> any authenticated user
[ ] Create AdminService with @PreAuthorize("hasRole('ADMIN')")
[ ] Test concurrent session control:
    [ ] Login as user1 in Chrome
    [ ] Login as user1 in Firefox (incognito)
    [ ] Verify: first session is expired/second login is blocked
[ ] Test CSRF: try a direct POST without _csrf token -> verify 403
[ ] Test Role-based access: user with ROLE_USER tries /admin -> verify 403
```

Exercise 4: Microservices Security with API Gateway (Advanced)

Goal

Build a microservices ecosystem with JWT-based auth passing through an API Gateway.

Step-by-Step

```
[ ] Create 3 Spring Boot projects: api-gateway, auth-service, order-service
[ ] Auth Service:
    [ ] POST /auth/login -> validates credentials, returns JWT (HS256 for
simplicity)
    [ ] Exposes JWKS endpoint (optional: hardcoded public key for simplicity)
[ ] API Gateway (Spring Cloud Gateway):
    [ ] Routes /api/auth/** -> auth-service
    [ ] Routes /api/orders/** -> order-service
    [ ] JwtGatewayFilter: validates JWT, forwards X-Forwarded-User header
[ ] Order Service:
    [ ] GET /api/orders -> returns orders for the user in X-Forwarded-User
header
    [ ] No Spring Security dependency (trusts Gateway)
[ ] Test flow:
    [ ] POST /api/auth/login -> get JWT
    [ ] GET /api/orders (with JWT) -> Gateway validates -> Order Service returns
data
    [ ] GET /api/orders (without JWT) -> Gateway returns 401
```

Exercise 5: Security Testing & Penetration Testing Basics

Goal

Test your secured application against common attack vectors.

Checklist

```
[ ] Test SQL Injection: Try login with email: ' OR 1=1 --
    -> Verify: Spring Data JPA prevents it (parameterized queries)
[ ] Test CSRF: Create an external HTML page that auto-submits a POST
    -> Verify: Spring rejects it (missing _csrf token)
[ ] Test XSS: Try injecting <script>alert('xss')</script> in input fields
    -> Verify: Content-Security-Policy header blocks inline scripts
[ ] Test Session Fixation: Copy JSESSIONID before login, try using it after
    -> Verify: JSESSIONID changes after authentication
[ ] Test Brute Force: Send 50 login requests with wrong passwords
    -> Verify: Account lockout kicks in after 5 attempts
[ ] Test CORS: Make a fetch() call from a different origin
    -> Verify: Browser blocks it (unless origin is in allowedOrigins)
[ ] Test JWT Tampering: Decode JWT, change "role" to "ADMIN", re-encode
```

```
    -> Verify: Signature validation fails (401 Unauthorized)
[ ] Test Expired Token: Wait for access token to expire, try using it
    -> Verify: 401 returned, refresh token flow generates new access token
```